House Price Prediction

About the Project:

The aim of this project is to predict the sale prices of houses based on various features and attributes provided in the dataset. This dataset, sourced from Kaggle, contains data collected from houses in the United States. The problem is a regression problem, where the goal is to estimate the SalePrice of a house based on a wide range of features, such as the house's physical characteristics, location, year built, and other relevant factors.

Problem Statement:

House price prediction plays a significant role in real estate analysis and decision-making. Accurate prediction models can assist buyers, sellers, and investors in making informed choices. This project focuses on developing a machine learning model to predict the sale prices of homes, leveraging features such as house size, location, and condition. The key objective is to build a predictive model that can accurately estimate the sale price of a house based on these characteristics.

Dataset Overview:

The dataset contains multiple attributes related to various house features. Below is a brief explanation of the columns included in the dataset:

- **Id**: A unique identifier for each house.
- **MSSubClass**: The type of dwelling involved in the sale.
- **MSZoning**: The general zoning classification of the sale.
- **LotFrontage**: The linear feet of street connected to the property.
- **LotArea**: Lot size in square feet.
- **Street**: Type of road access to the property (e.g., Pave).
- **Alley**: Type of alley access (if applicable).
- **LotShape**: The general shape of the property.
- **LandContour**: The flatness or slope of the property.
- **Utilities**: Type of utilities available (e.g., AllPub for all public utilities).
- **LotConfig**: Configuration of the lot (e.g., Inside, Corner, etc.).
- **LandSlope**: The slope of the land (e.g., Gtl for Gentle).
- **Neighborhood**: The physical locations within the city or town.
- **Condition1 & Condition2**: Proximity to various conditions such as roads or railways.
- **BldgType**: Type of building (e.g., Single-family, Duplex).
- **HouseStyle**: Style of dwelling (e.g., 1-story, 2-story).
- **OverallQual & OverallCond**: Overall material and finish quality, and condition of the house.
- **YearBuilt & YearRemodAdd**: Year the house was built and remodeled.
- **RoofStyle & RoofMatl**: Type of roof and material used.
- **Exterior1st & Exterior2nd**: Primary and secondary exterior materials.
- **MasVnrType & MasVnrArea**: Type and area of masonry veneer.

- **BsmtQual, BsmtCond, BsmtExposure, BsmtFinType1, BsmtFinSF1, etc.**: Basement-related features, including quality, condition, exposure, and finished area.
- **Heating & HeatingQC**: Type and quality of heating system.
- **CentralAir**: Whether the house has central air conditioning.
- **Electrical**: Type of electrical system.
- **1stFlrSF & 2ndFlrSF**: Square footage of the first and second floors.
- **GrLivArea**: Above-ground living area in square feet.
- **GarageType, GarageYrBlt, GarageFinish, GarageCars, GarageArea**: Garage-related features.
- **PavedDrive**: Type of driveway (e.g., Pave, Grvl).
- **MiscFeature & MiscVal**: Miscellaneous features and their values.
- **MoSold & YrSold**: Month and year the house was sold.
- **SaleType & SaleCondition**: Type and condition of sale.

Target Variable:

- **SalePrice**: The price of the house in USD, which is the dependent variable for prediction.

This dataset provides a comprehensive set of features that can be used to analyze housing prices and develop predictive models. The project will involve cleaning the data, exploring patterns and relationships, and building a robust machine learning model to predict house prices accurately.

Content:

1. **Exploratory Data Analysis.**
2. **Data Preprocessing.**
3. **Model building & Model performance evaluation.**
    a. **ML-1 Techniques**
        i. **Linear regression model.**
        ii. **Polynomial Regression.**
        iii. **Decision Tree.**
        iv. **Random Forest.**
        v. **Boosting.**
    b. **ML-2 Techniques & Trying to Improve RMSE.**
        i. **Principal Component Analysis.**
        ii. **T-SNE(t-distributed Stochastic Neighbor Embedding)**
        iii. **LOF (Local Outlier Factor) and Isolation Forest**
        iv. **K-Means**
        v. **Hierarchical Clustering.**
        vi. **DBSCAN**
        vii. **EDA on Clusters.**
        viii. **Target transformation**
        ix. **Cluster Vise Model**
4. **Actionable Insights & Recommendations.**

1. Exploratory Data Analysis.

Library used

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12, 10)

from sklearn.metrics import r2_score

from sklearn.metrics import mean_squared_error as mse

from sklearn.ensemble import GradientBoostingRegressor

from sklearn.model_selection import KFold

from sklearn.metrics import mean_squared_error as mse

from sklearn.metrics import mean_absolute_percentage_error as mape

from sklearn.model_selection import train_test_split

from sklearn.model_selection import cross_val_score

import pandas as pd

from sklearn.impute import SimpleImputer as Imputer

from random import choices

from sklearn.preprocessing import StandardScaler, MinMaxScaler

from sklearn.preprocessing import LabelEncoder

from sklearn.linear_model import LinearRegression

from sklearn.svm import LinearSVR

from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.ensemble import GradientBoostingRegressor

from sklearn.neighbors import KNeighborsRegressor

from sklearn.ensemble import StackingRegressor

from sklearn.base import BaseEstimator
```

Data

```
!gdown 1F9aD344ifJiKzRDlR4ZX0e4plDuP9-b9
```

```
Downloading...
From: https://drive.google.com/uc?id=1F9aD344ifJiKzRDlR4ZX0e4plDuP9-b9
To: /content/kaggle_houseprices_modified.csv
100% 459k/459k [00:00<00:00, 19.4MB/s]
```

```
ds = pd.read_csv('/content/kaggle_houseprices_modified.csv')

ds.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD

5 rows × 81 columns

```
ds.columns
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage', 'LotArea', 'Street',
       'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig',
       'LandSlope', 'Neighborhood', 'Condition1', 'Condition2', 'BldgType',
       'HouseStyle', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
       'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd', 'MasVnrType',
       'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation', 'BsmtQual',
       'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinSF1',
       'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'Heating',
       'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF', '2ndFlrSF',
       'LowQualFinSF', 'GrLivArea', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath',
       'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'KitchenQual',
       'TotRmsAbvGrd', 'Functional', 'Fireplaces', 'FireplaceQu', 'GarageType',
       'GarageYrBlt', 'GarageFinish', 'GarageCars', 'GarageArea', 'GarageQual',
       'GarageCond', 'PavedDrive', 'WoodDeckSF', 'OpenPorchSF',
       'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'PoolQC',
       'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold', 'SaleType',
       'SaleCondition', 'SalePrice'],
      dtype='object')
```

```
ds[ds.columns[80:]].head()
```

	SalePrice
0	198075.0
1	199650.0
2	212325.0
3	140000.0
4	237500.0

Insights from Exploratory Data Analysis (EDA)

1. **Dataset Overview**: The dataset contains **81 columns** with a mix of categorical, numerical, and object features. The target variable is **SalePrice**.
2. **Missing Values**: Several columns have missing values, especially **Alley**, **PoolQC**, **Fence**, **MiscFeature**, and basement-related columns (**BsmtQual**, **BsmtCond**, etc.). Imputation techniques will be needed to handle these gaps.
3. **Categorical Variables**: Features like **MSZoning**, **Street**, **Neighborhood**, and **GarageType** are categorical. These will need encoding for machine learning models, with some having many unique values.
4. **Outliers**: Outliers were found in **LotFrontage**, **LotArea**, **GrLivArea**, and **SalePrice**, which may need capping or removal to improve model accuracy.
5. **Skewed Distributions**: **SalePrice** and **LotArea** are highly skewed. Applying **log transformation** could help normalize these features for better model performance.
6. **Correlation with SalePrice**: **GrLivArea**, **OverallQual**, and **GarageArea** show strong correlations with **SalePrice**, indicating their importance in predicting house prices.
7. **Data Type Issues**: Some columns are misclassified (e.g., numerical treated as categorical). These need to be corrected for proper analysis.
8. **Feature Engineering**: Temporal features like **YearBuilt** and **GarageYrBlt** could be transformed into new features like **Age of the House** or **Years Since Remodeled**.
9. **Multicollinearity**: Variables like **TotRmsAbvGrd** and **GrLivArea** are highly correlated. Reducing multicollinearity will be necessary, possibly through feature removal or PCA.

2. Data Preprocessing:

```
todrop = ['Id']
categorical = ['MSSubClass', 'MSZoning', 'Street', 'Alley',
'LotShape', 'LandContour', 'Utilities', 'LotConfig',
               'LandSlope', 'Neighborhood', 'Condition1',
'Condition2', 'BldgType', 'RoofStyle', 'RoofMatl', 'HouseStyle',
               'Exterior1st', 'Exterior2nd', 'MasVnrType',
'Foundation', 'Heating', 'CentralAir',
               'Electrical', 'GarageType', 'PavedDrive', 'Fence',
'MiscFeature', 'MoSold', 'SaleType', 'SaleCondition']
continuous = ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond',
'YearBuilt', 'YearRemodAdd', 'BsmtFinSF1',
              'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', '1stFlrSF',
'2ndFlrSF', 'LowQualFinSF', 'GrLivArea', 'MasVnrArea',
              'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd',
              'Fireplaces', 'GarageYrBlt', 'GarageCars',
'GarageArea', 'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch',
              '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal',
'YrSold', 'SalePrice']
cat_to_con = ['ExterQual', 'ExterCond', 'BsmtQual', 'BsmtCond',
'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
              'HeatingQC', 'KitchenQual', 'Functional',
'FireplaceQu', 'GarageFinish', 'GarageQual', 'GarageCond',
              'PoolQC', ]
```

```
# run the imputation when not making pipeline

ds['LotFrontage'] = ds['LotFrontage'].fillna(ds['LotFrontage'].median())

ds['MasVnrArea']  = ds['MasVnrArea'].fillna(ds['MasVnrArea'].median())

ds['GarageYrBlt'] = ds['GarageYrBlt'].fillna(ds['GarageYrBlt'].mean())
```

```
print(ds[continuous].isna().sum().sum())

def pre_processing(ds, todrop, categorical, cat_to_con):

  #dropping

  ds = ds.drop(columns=todrop)

  #categorical

  logical_nans = ['Alley', 'Fence', 'MiscFeature', 'MasVnrType',
'GarageType', ]

  for col in logical_nans:

      ds[col] = ds[col].fillna('NA')

  ds['Electrical'] =
ds['Electrical'].fillna(ds['Electrical'].value_counts().index[0])  # mode

  print("no. of nans in categorical cols:",
ds[categorical].isna().sum().sum())

  #cat_to_con

  ds['FireplaceQu'] = ds['FireplaceQu'].fillna('NA')

  logical_nans = ['FireplaceQu', 'BsmtQual', 'BsmtCond', 'BsmtExposure',
'BsmtFinType1',

                  'BsmtFinType2', 'GarageFinish', 'GarageQual',
'GarageCond']

  for col in logical_nans:

      ds[col] = ds[col].fillna('NA')

  ds.drop(columns = ['PoolQC'], inplace=True)

  cat_to_con.remove('PoolQC')

  print("no. of nans in cat_to_con
cols:",ds[cat_to_con].isna().sum().sum())
```

```
#target_encoding

  target = 'SalePrice'

  for col in categorical:

      ds[col] = ds.groupby([col])[target].transform('mean'

  #label encoding

  grp1 = ['GarageQual', 'GarageCond', 'FireplaceQu', 'KitchenQual',
'HeatingQC', 'BsmtCond', 'BsmtQual', 'ExterCond', 'ExterQual']

  for col in grp1:

      ds[col] = ds[col].map({'TA': 3, 'Fa': 2, 'NA': 0, 'Gd': 4, 'Po': 1,
'Ex': 5})

  for col in ['BsmtFinType1', 'BsmtFinType2']:

      ds[col] = ds[col].map({'GLQ':6, 'ALQ':5, 'Unf':1, 'Rec':3, 'BLQ':4,
'NA':0, 'LwQ':2})

  ds['BsmtExposure'] = ds['BsmtExposure'].map({'No':1, 'Gd':4, 'Mn':2,
'Av':3, 'NA':0})

  ds['Functional'] = ds['Functional'].map({'Typ':7, 'Min1':6, 'Maj1':3,
'Min2':5, 'Mod':4, 'Maj2':2, 'Sev':1, 'Sal':0})

  ds['GarageFinish'] = ds['GarageFinish'].map({'RFn':2, 'Unf':1, 'Fin':3,
'NA':0})

  print("total nans in the dataframe:", ds.isna().sum().sum())

  return ds
```

```
ds = pre_processing(ds, todrop, categorical, cat_to_con)
```

```
no. of nans in categorical cols: 0
no. of nans in cat_to_con cols: 0
total nans in the dataframe: 0
```

ds

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	ScreenPorch	PoolArea	F
0	252692.601839	204994.032276	65.0	8450	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	201832.73
1	207766.905970	204994.032276	80.0	9600	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	178829.893617	...	0	0	201832.73
2	252692.601839	204994.032276	68.0	11250	194047.191231	196590.593755	219936.183574	193292.650229	193813.919842	190814.936930	...	0	0	201832.73
3	174123.741667	204994.032276	60.0	9550	194047.191231	196590.593755	219936.183574	193292.650229	193813.919842	193082.861217	...	0	0	201832.73
4	252692.601839	204994.032276	84.0	14260	194047.191231	196590.593755	219936.183574	193292.650229	193813.919842	178829.893617	...	0	0	201832.73
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1455	252692.601839	204994.032276	62.0	7917	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	201832.73
1456	207766.905970	204994.032276	85.0	13175	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	155835.05
1457	174123.741667	204994.032276	66.0	9042	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	182889.57
1458	207766.905970	204994.032276	68.0	9717	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	201832.73
1459	207766.905970	204994.032276	75.0	9937	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0	0	201832.73

1460 rows × 79 columns

ds.describe()

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	LotConfig	...	ScreenPorch	PoolArea
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	...	1460.000000	1460.000000
mean	193784.766473	193784.766473	69.863699	10516.828082	193784.766473	193784.766473	193784.766473	193784.766473	193784.766473	193784.766473	...	15.060959	2.758904
std	45437.758786	31822.387064	22.027677	9981.264932	4086.581515	13232.392569	23002.475340	15534.371834	1113.948694	11337.630486	...	55.757415	40.177307
min	91550.000000	73276.870000	21.000000	1300.000000	130190.500000	124300.488000	176882.391676	147545.992063	151250.000000	178829.893617	...	0.000000	0.000000
25%	150141.173611	204994.032276	60.000000	7553.500000	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0.000000	0.000000
50%	207766.905970	204994.032276	69.000000	9478.500000	194047.191231	196590.593755	176882.391676	193292.650229	193813.919842	190814.936930	...	0.000000	0.000000
75%	218035.000000	204994.032276	79.000000	11601.500000	194047.191231	196590.593755	219936.183574	193292.650229	193813.919842	190814.936930	...	0.000000	0.000000
max	252692.601839	248375.383077	313.000000	215245.000000	194047.191231	196590.593755	255879.595122	259341.542000	193813.919842	235935.562766	...	480.000000	738.000000

8 rows × 79 columns

Insights on the Preprocessing Function:

1. **Dropping Unnecessary Columns**: The function begins by removing columns that are not necessary for model training, such as the **'Id'** column, which is a unique identifier that doesn't contribute to predicting house prices. The `todrop` parameter allows you to specify other columns that are irrelevant for the analysis.
2. **Handling Missing Values in Categorical Columns**: For categorical features that may contain missing values, like **'Alley'**, **'Fence'**, **'MiscFeature'**, **'MasVnrType'**, and **'GarageType'**, you fill missing entries with a placeholder value `'NA'`. This step ensures that the model won't break due to missing data and avoids the need for dropping these columns.
    - Additionally, for the **'Electrical'** column, missing values are filled with the mode (the most frequent value) of the column. This is a common strategy when dealing with categorical variables, as it ensures that the majority class is represented and doesn't significantly affect model performance.
3. **Handling Missing Values in `cat_to_con` Columns**: The function processes columns that are a mix of categorical and continuous variables (referred to as `cat_to_con`). Missing values in these columns, such as **'FireplaceQu'**, **'BsmtQual'**, **'BsmtCond'**, **'BsmtExposure'**, and others, are also filled with `'NA'`. For **'PoolQC'**, the column is entirely dropped because it has a high proportion of missing values and may not provide useful information for the model.
4. **Target Encoding for Categorical Features**: One of the key transformations in your preprocessing pipeline is **target encoding**, where you replace each category in categorical columns with the mean of the target variable (i.e., **SalePrice**) for that category. This approach captures the relationship between the categorical feature and the target variable, and it can improve predictive performance by incorporating information about how each category correlates with house prices.
   For example, for a categorical variable like **'MSZoning'**, target encoding will replace each zone (e.g., 'RL', 'RM') with the mean **SalePrice** for houses in that zone. This step helps capture trends and is particularly useful for models like decision trees, which can take advantage of this encoded information.
5. **Label Encoding for Ordinal Features**: **Label encoding** is applied to features with an ordinal relationship (where categories have a natural order), such as **'ExterCond'**, **'FireplaceQu'**, and others. In these cases, you manually map each category to an integer that reflects its rank or quality:
    - For example, in **'ExterCond'** (Exterior Condition), categories like `'Ex'` (Excellent), `'Gd'` (Good), `'TA'` (Typical), and others are mapped to integers from 1 to 5, where a higher number represents better quality.
6. Similarly, columns like **'BsmtFinType1'** and **'GarageFinish'** are also label-encoded, with each category being assigned a specific number to represent its level of finish.
   This transformation is critical because machine learning algorithms often work better with numerical inputs, and label encoding allows you to represent the ordinal nature of these features while preserving the order of values.
7. **Final Check for Missing Values**: After performing all imputation, encoding, and dropping operations, the function prints out the total number of missing values in the entire dataset to ensure that no missing data remains. This step is crucial before moving on to modeling because most algorithms require a complete dataset.

3. Model building & Model performance evaluation.

A. ML-1 Techniques:

i. Linear Regression:

```
X = ds_scaled[:, :-1]

y = ds['SalePrice'].values
X.shape, y.shape
```

```
((1460, 78), (1460,))
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

X_train.shape, X_test.shape , y_train.shape, y_test.shape
```

```
((1168, 78), (292, 78), (1168,), (292,))
```

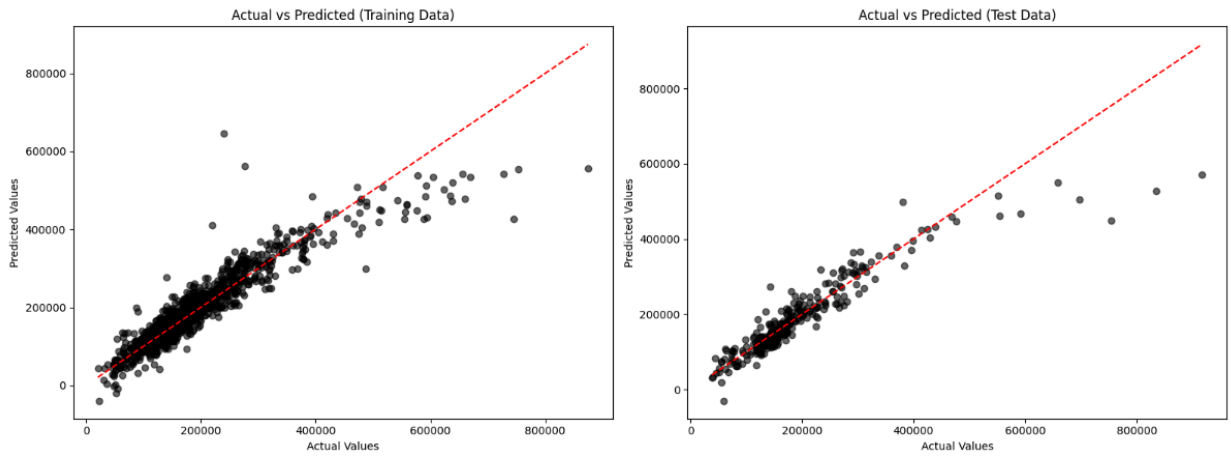
```
# Linear regression [implementation]

from sklearn.linear_model import LinearRegression

model1 = LinearRegression()

model1.fit(X_train, y_train) # training the model
```

```
LinearRegression
LinearRegression()
```

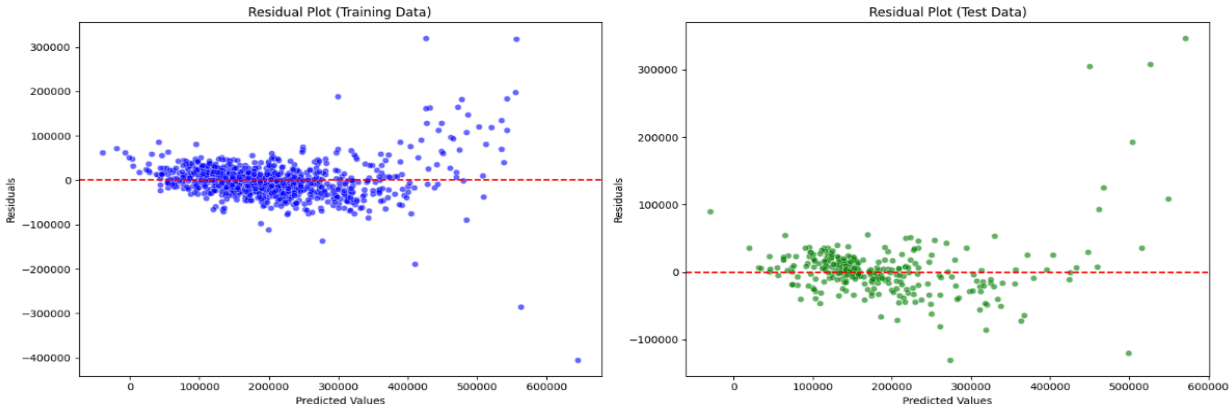
```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Predict for training and testing sets

y_pred_train = model.predict(X_train)

y_pred_test = model.predict(X_test)

# Evaluate the model performance on both training and testing data

# Calculate metrics for training data

mse_train = mean_squared_error(y_train, y_pred_train)

mae_train = mean_absolute_error(y_train, y_pred_train)

r2_train = r2_score(y_train, y_pred_train)

# Calculate metrics for test data

mse_test = mean_squared_error(y_test, y_pred_test)

mae_test = mean_absolute_error(y_test, y_pred_test)

r2_test = r2_score(y_test, y_pred_test)

# Print evaluation metrics

print("Training Data Metrics:")

print(f"MSE: {mse_train:.4f}, MAE: {mae_train:.4f}, R²: {r2_train:.4f}")

print("\nTest Data Metrics:")

print(f"MSE: {mse_test:.4f}, MAE: {mae_test:.4f}, R²: {r2_test:.4f}")
```

```
Training Data Metrics:
MSE: 1446595241.2733, MAE: 23269.3293, R²: 0.8562

Test Data Metrics:
MSE: 2028598486.7990, MAE: 24417.3631, R²: 0.8520
```

```
# Visualize Actual vs Predicted.

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training data plot

axes[0].scatter(y_train, y_pred_train, color='black', alpha=0.6)

axes[0].plot([min(y_train), max(y_train)], [min(y_train), max(y_train)],
color='yellow', linestyle='--')  # Perfect prediction line

axes[0].set_title('Actual vs Predicted (Training Data)')

axes[0].set_xlabel('Actual Values')

axes[0].set_ylabel('Predicted Values')

# Test data plot

axes[1].scatter(y_test, y_pred_test, color='black', alpha=0.6)

axes[1].plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
color='yellow', linestyle='--')  # Perfect prediction line

axes[1].set_title('Actual vs Predicted (Test Data)')

axes[1].set_xlabel('Actual Values')

axes[1].set_ylabel('Predicted Values')

plt.tight_layout()  # Adjust layout to prevent overlap
```



```
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

residuals_train = y_train - y_pred_train

sns.scatterplot(x=y_pred_train, y=residuals_train, color='blue', alpha=0.6,
ax=axes[0])

axes[0].axhline(0, color='red', linestyle='--')

axes[0].set_title('Residual Plot (Training Data)')

axes[0].set_xlabel('Predicted Values')

axes[0].set_ylabel('Residuals')

# Test residuals plot

residuals_test = y_test - y_pred_test

sns.scatterplot(x=y_pred_test, y=residuals_test, color='green', alpha=0.6,
ax=axes[1])

axes[1].axhline(0, color='red', linestyle='--')

axes[1].set_title('Residual Plot (Test Data)')

axes[1].set_xlabel('Predicted Values')

axes[1].set_ylabel('Residuals')

plt.tight_layout()  # Adjust layout to prevent overlap
```



ii. Polynomial Regression.

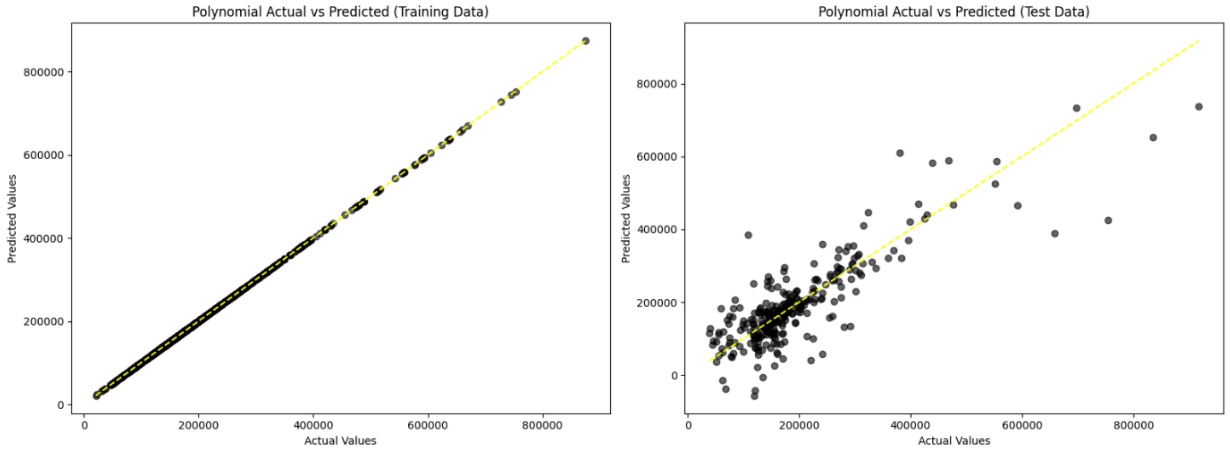
```
from sklearn.preprocessing import PolynomialFeatures

# Create Polynomial features

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)

X_test_poly = poly.transform(X_test)

# Train the Polynomial Regression model

poly_model = LinearRegression()

poly_model.fit(X_train_poly, y_train)

# Predict for training and testing sets

y_pred_train_poly = poly_model.predict(X_train_poly)

y_pred_test_poly = poly_model.predict(X_test_poly)

# Evaluate the Polynomial model performance

mse_train_poly = mean_squared_error(y_train, y_pred_train_poly)

mae_train_poly = mean_absolute_error(y_train, y_pred_train_poly)

r2_train_poly = r2_score(y_train, y_pred_train_poly)

mse_test_poly = mean_squared_error(y_test, y_pred_test_poly)

mae_test_poly = mean_absolute_error(y_test, y_pred_test_poly)

r2_test_poly = r2_score(y_test, y_pred_test_poly)

# Print evaluation metrics

print("Polynomial Regression - Training Data Metrics:")

print(f"MSE: {mse_train_poly:.4f}, MAE: {mae_train_poly:.4f}, R²:
{r2_train_poly:.4f}")

print("\nPolynomial Regression - Test Data Metrics:")
```

```
print(f"MSE: {mse_test_poly:.4f}, MAE: {mae_test_poly:.4f}, R²:
{r2_test_poly:.4f}")
```

```
Polynomial Regression - Training Data Metrics:
MSE: 0.0000, MAE: 0.0000, R²: 1.0000

Polynomial Regression - Test Data Metrics:
MSE: 4198431641.3503, MAE: 42743.7889, R²: 0.6936
```

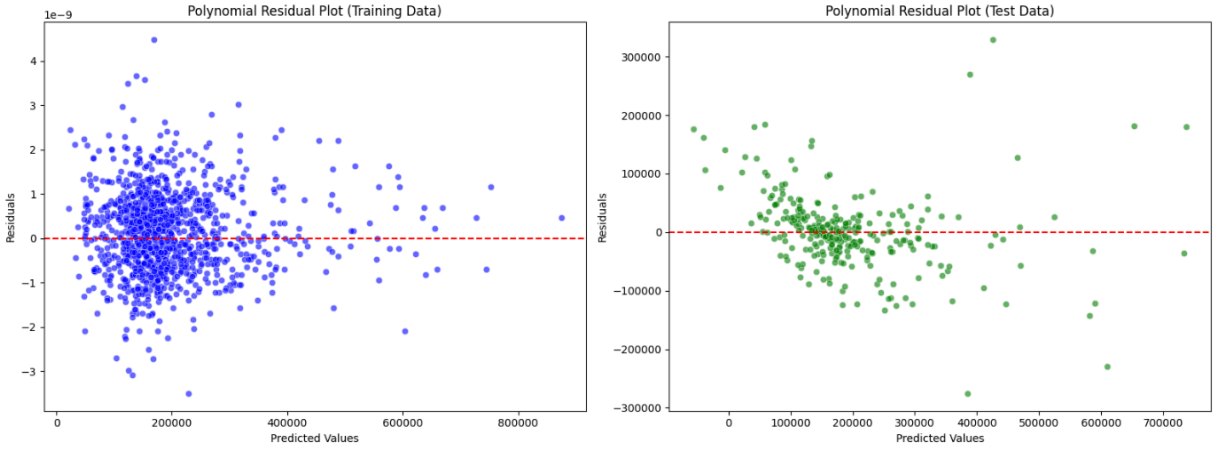
```
# Visualize Actual vs Predicted (Side by Side)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training data plot (Polynomial Regression)

axes[0].scatter(y_train, y_pred_train_poly, color='black', alpha=0.6)

axes[0].plot([min(y_train), max(y_train)], [min(y_train), max(y_train)],
color='yellow', linestyle='--')  # Perfect prediction line

axes[0].set_title('Polynomial Actual vs Predicted (Training Data)')

axes[0].set_xlabel('Actual Values')

axes[0].set_ylabel('Predicted Values')

# Test data plot (Polynomial Regression)

axes[1].scatter(y_test, y_pred_test_poly, color='black', alpha=0.6)

axes[1].plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
color='yellow', linestyle='--')  # Perfect prediction line

axes[1].set_title('Polynomial Actual vs Predicted (Test Data)')

axes[1].set_xlabel('Actual Values')

axes[1].set_ylabel('Predicted Values')

plt.tight_layout()  # Adjust layout to prevent overlap
```

```
plt.show()
```




```
# Residuals Plot (Side by Side)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Training residuals plot (Polynomial Regression)

residuals_train_poly = y_train - y_pred_train_poly

sns.scatterplot(x=y_pred_train_poly, y=residuals_train_poly, color='blue',
alpha=0.6, ax=axes[0])

axes[0].axhline(0, color='red', linestyle='--')

axes[0].set_title('Polynomial Residual Plot (Training Data)')

axes[0].set_xlabel('Predicted Values')

axes[0].set_ylabel('Residuals')

# Test residuals plot (Polynomial Regression)

residuals_test_poly = y_test - y_pred_test_poly

sns.scatterplot(x=y_pred_test_poly, y=residuals_test_poly, color='green',
alpha=0.6, ax=axes[1])
```

```
axes[1].axhline(0, color='red', linestyle='--')

axes[1].set_title('Polynomial Residual Plot (Test Data)')

axes[1].set_xlabel('Predicted Values')

axes[1].set_ylabel('Residuals')

plt.tight_layout()  # Adjust layout to prevent overlap

plt.show()
```



Insights on Linear and Polynomial Regression Model:

Linear Regression Performance:

- The **Linear Regression model** demonstrates strong predictive performance, achieving an **R^2 of 0.8562** on the training data and **0.8520** on the test data. This indicates that the model explains about 85% of the variance in the target variable, even on unseen data, which reflects good generalization.
- However, the **Mean Absolute Error (MAE)** of **24417.36** on the test data suggests that the model's predictions deviate by an average of around 24,417 units from the actual values. This error, while not extremely high, could still be reduced to improve accuracy.

Polynomial Regression Performance:

- The **Polynomial Regression model** achieves a perfect fit (**$R^2 = 1.0000$**) on the training data, suggesting overfitting. The model captures all the variance in the training set but fails to generalize well to the test data.
- On the test set, the **R^2 drops to 0.6936**, and the **MAE increases to 42,743.79**, which is significantly worse than the Linear Regression model. This indicates that the complexity of the Polynomial model causes it to perform poorly on unseen data, likely due to overfitting.

Conclusion:

- The Linear Regression model outperforms the Polynomial Regression model in terms of generalization to test data, making it the better choice between the two for this dataset.
- The Polynomial Regression model's poor performance on test data highlights the risk of overfitting when increasing model complexity without careful validation.

Next Steps Decision Tree and Random Forest models

To further evaluate and potentially improve model performance, **let's try implementing Decision Tree and Random Forest models**. These models can capture nonlinear relationships without the risk of overfitting as heavily as Polynomial Regression if properly tuned. Here's why:

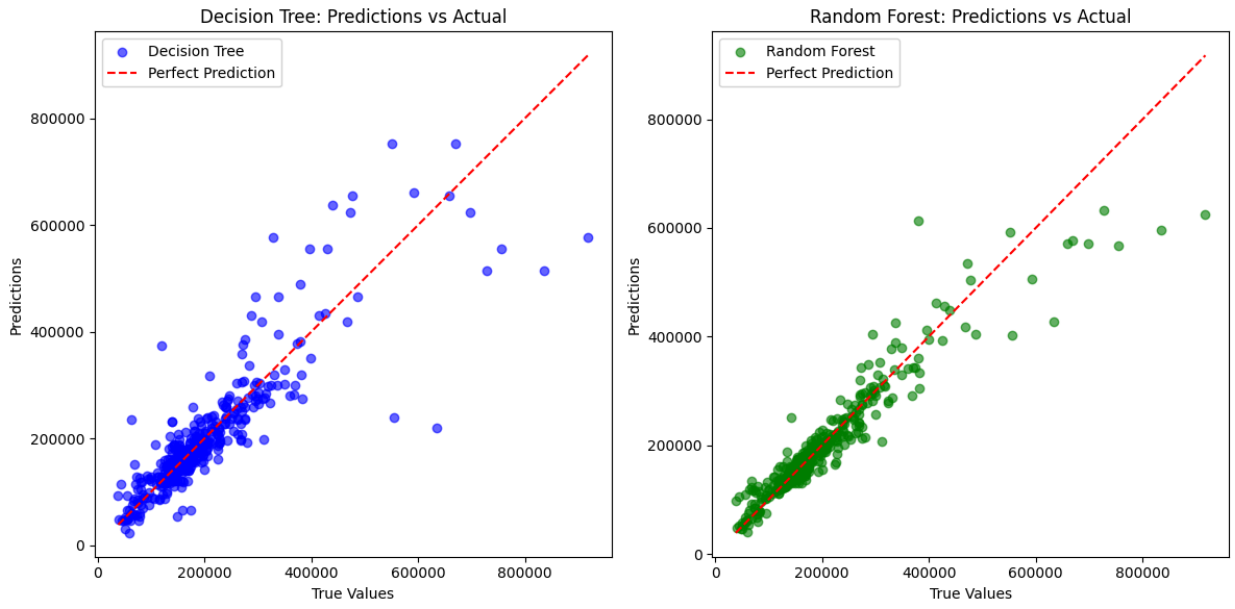
- **Decision Tree**: Captures complex patterns in the data but may overfit; pruning or hyperparameter tuning is necessary.
- **Random Forest**: A robust ensemble model that reduces overfitting and provides feature importance insights.

iii. Decision Tree & Random Forest:

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import accuracy_score, classification_report

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Initialize the models
decision_tree = DecisionTreeRegressor(random_state=42)
random_forest = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the models on the non-scaled training data
decision_tree.fit(X_train, y_train)
random_forest.fit(X_train, y_train)

# Predict using both models
y_pred_tree = decision_tree.predict(X_test)
y_pred_forest = random_forest.predict(X_test)

# Calculate Mean Squared Error (MSE) and R² for both models
mse_tree = mean_squared_error(y_test, y_pred_tree)
mse_forest = mean_squared_error(y_test, y_pred_forest)
r2_tree = r2_score(y_test, y_pred_tree)
r2_forest = r2_score(y_test, y_pred_forest)

print(f"Decision Tree - MSE: {mse_tree:.2f}, R²: {r2_tree:.2f}")
print(f"Random Forest - MSE: {mse_forest:.2f}, R²: {r2_forest:.2f}")
```

```
Decision Tree - MSE: 3183012964.95, R²: 0.75
Random Forest - MSE: 1333411556.13, R²: 0.90
```

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred_tree, color='blue', alpha=0.6, label='Decision
Tree')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
color='red', linestyle='--', label='Perfect Prediction')
plt.title('Decision Tree: Predictions vs Actual')
plt.xlabel('True Values')
plt.ylabel('Predictions')
plt.legend()
# Random Forest
plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_forest, color='green', alpha=0.6, label='Random
Forest')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)],
color='red', linestyle='--', label='Perfect Prediction')
plt.title('Random Forest: Predictions vs Actual')
plt.xlabel('True Values')
plt.ylabel('Predictions')
plt.legend()
plt.tight_layout()
```



Compare the Models:

```
# Compare the models based on error metrics
comparison = pd.DataFrame({
    'Model': ['Decision Tree', 'Random Forest'],
    'MSE': [mse_tree, mse_forest],
    'R²': [r2_tree, r2_forest]
})
print("\nModel Comparison:")
print(comparison)
```

```
Model Comparison:
           Model           MSE        R²
0  Decision Tree  3.183013e+09  0.750516
1  Random Forest  1.333412e+09  0.895487
```

Insights on Decision Tree and Random Forest Models:

Decision Tree Performance:

- The Decision Tree model achieves an R^2 of **0.7505** on the test data, meaning it explains 75% of the variance in the target variable. While this is a reasonable performance, it is relatively lower than that of both the Linear Regression and Random Forest models.
- The Mean Squared Error (MSE) of **3.183×10^9** suggests a higher average squared error compared to the other models, which indicates less precision in the predictions.
- Decision Trees are prone to overfitting, which can lead to good performance on training data but poorer generalization on unseen data.

Random Forest Performance:

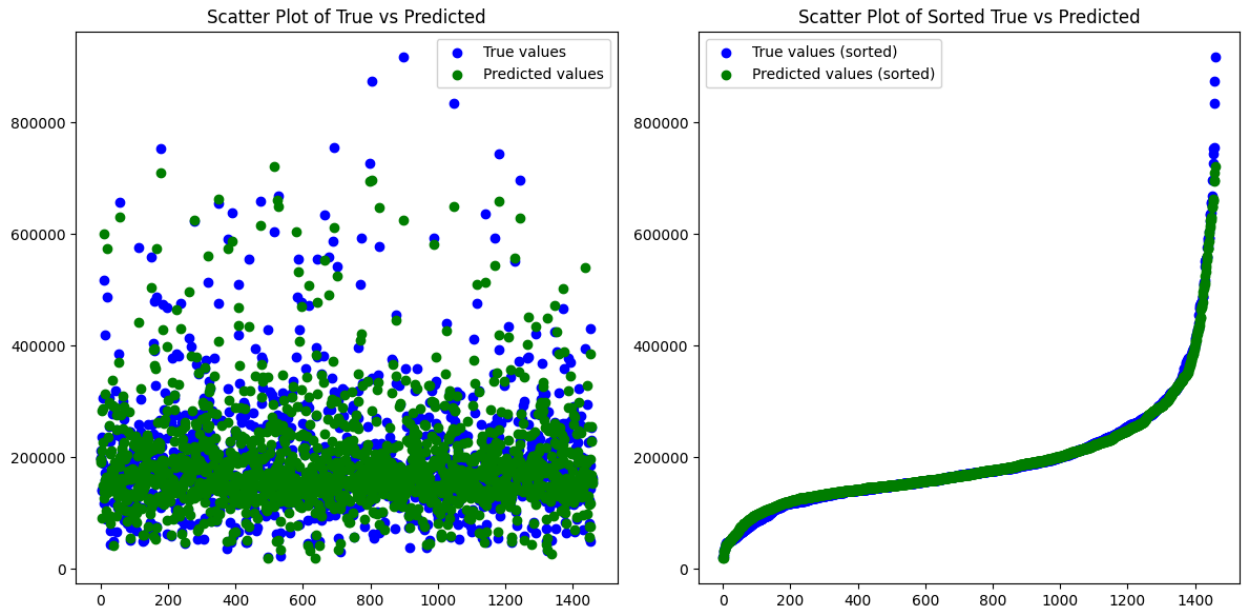
- The Random Forest model significantly outperforms the Decision Tree, with an R^2 of **0.8955** on the test data, indicating it explains nearly 90% of the variance in the target variable. This is the best performance among the models tested.
- With an MSE of **1.333×10^9**, the Random Forest model's predictions are much more accurate than those of the Decision Tree, suggesting a higher level of generalization and less overfitting.
- Random Forests combine the predictions of multiple decision trees, which helps reduce the variance and improve robustness.

iv. Boosting Model:

```
# Initialize KFold cross-validation with 5 splits
kf = KFold(n_splits=5)
# Empty arrays to store true and predicted values for later evaluation
y_true, y_pred = np.array([]), np.array([])
# Loop over each split in the cross-validation
for train_index, test_index in kf.split(X):

    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    # Initialize a Gradient Boosting Regressor model with a fixed random
seed for reproducibility
    baseline_estimator = GradientBoostingRegressor(random_state=0)

    # Fit the model to the training data
    baseline_estimator.fit(X_train, y_train)

    # Append the true values (y_test) and predicted values (from model's
predictions on X_test)
    # to the previously initialized arrays to collect results from all
folds
    y_true = np.append(y_true, y_test)
    y_pred = np.append(y_pred, baseline_estimator.predict(X_test))

rmse = mse(y_true, y_pred)**0.5
print(f"Root Mean Squared Error (RMSE): {rmse}")

mape_value = mape(y_true, y_pred)
print(f"Mean Absolute Percentage Error (MAPE): {mape_value}")
```

```
Root Mean Squared Error (RMSE): 31277.140430213938
Mean Absolute Percentage Error (MAPE): 0.10184115406028563
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
# First plot: Scatter plot of true values (y_true) & predicted values
(y_pred)
axes[0].scatter(range(len(y_true)), y_true, label='True values',
color='blue')
axes[0].scatter(range(len(y_true)), y_pred, label='Predicted values',
color='green')
axes[0].set_title('Scatter Plot of True vs Predicted')
axes[0].legend()
# Second plot: Scatter plot of true values (y_true) & predicted values
(y_pred) in sorted form
axes[1].scatter(range(len(y_true)), sorted(y_true), label='True values
(sorted)', color='blue')
axes[1].scatter(range(len(y_true)), sorted(y_pred), label='Predicted
values (sorted)', color='green')
axes[1].set_title('Scatter Plot of Sorted True vs Predicted')
axes[1].legend()
# Adjust the layout to make it look better
plt.tight_layout()
plt.show()
```



Insights on Boosting Model:

Boosting Performance:

- **Root Mean Squared Error (RMSE):** The Boosting model has an RMSE of **31,277.14**, which indicates that, on average, the model's predictions deviate by approximately 31,277 units from the actual values. This error is higher than the Random Forest's MSE but lower than the Polynomial Regression's MAE, indicating a relatively moderate level of prediction error.
- **Mean Absolute Percentage Error (MAPE):** The Boosting model achieves a **MAPE of 10.18%**, meaning the model's predictions deviate from the actual values by an average of 10.18% in absolute terms. This is a reasonable level of accuracy, with lower values indicating better performance. The MAPE is comparable to the Linear and Random Forest models, showing good performance in terms of percentage error.

Supervised Models Conclusion:

- **Best Performing Model**: The **Random Forest** remains the top performer with the highest R^2 (0.8955) and lowest MSE (1.333×10^9), followed by **Linear Regression** which provides a reliable, interpretable, and efficient solution.
- **Moderate Performance**: **Boosting** presents competitive performance with a **MAPE of 10.18%** and **RMSE of 31,277.14**, indicating that while it doesn't outperform Random Forest, it still provides good generalization and predictive power.
- **Decision Tree**: The **Decision Tree** model, with a higher MSE (3.183×10^9) and lower R^2 (0.7505), remains prone to overfitting, providing less accurate predictions than the more robust models.
- **Underperforming**: The **Polynomial Regression** model continues to underperform due to its overfitting issue, as highlighted by its poor performance on the test data (R^2 = 0.6936).

In conclusion, **Random Forest** still provides the most accurate predictions with the best generalization, while **Boosting** offers a solid alternative with good error metrics. Both models outperform **Decision Tree** and **Polynomial Regression** in terms of predictive accuracy.

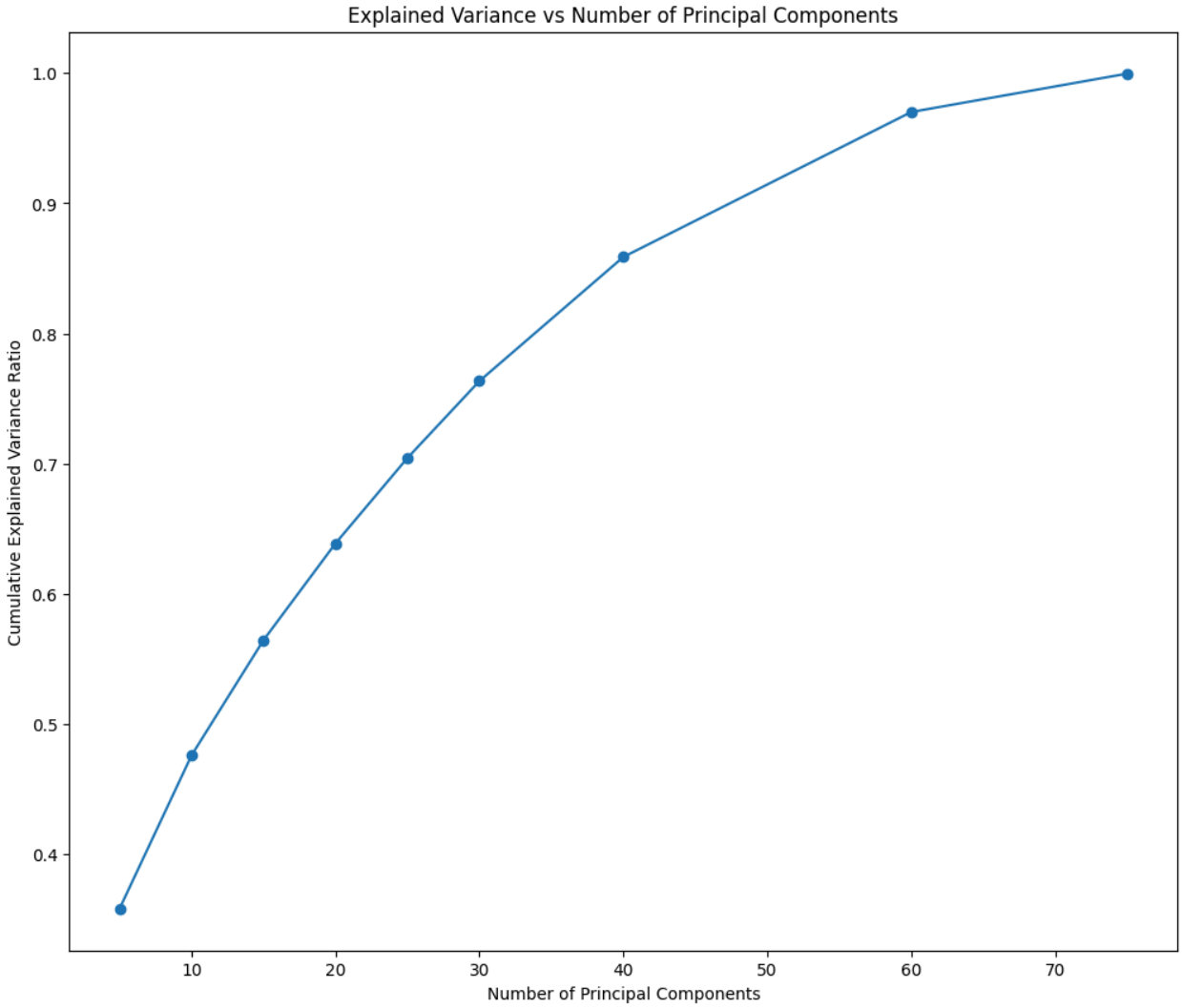
B. ML-2 Techniques & Trying to Improve RMSE.

i. Principal Component Analysis:

```
from sklearn.decomposition import PCA  # Import the PCA class from sklearn

# List of different values for the number of principal components to test
cs = [5, 10, 15, 20, 25, 30, 40, 60, 75]

info = []

for i in cs:
    pca = PCA(n_components=i)
    pca.fit(ds_scaled[:, :-1])
    info.append(pca.explained_variance_ratio_.sum())

info
```

```
[0.35819701075632565,
 0.4759750529468739,
 0.5644949413138924,
 0.6388056295631945,
 0.7044158157422951,
 0.7635118096335104,
 0.8588193310788007,
 0.9701672289544914,
 0.9996492681089242]
```

```
# '-o' specifies a line plot with circular markers at each data point
plt.plot(cs, info, '-o')
plt.title("Explained Variance vs Number of Principal Components")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance Ratio")
plt.show()
```



```
# Initialize PCA with 50 principal components
pca = PCA(n_components=50)
Xpca = pca.fit_transform(ds_scaled[:, :-1])
Xpca.shape
```

```
(1460, 50)
```

```
# Initialize KFold cross-validation with 5 splits
kf = KFold(n_splits=5)
mses = []
mapes = []
n = 0
y_true, y_pred = np.array([]), np.array([])
# Perform cross-validation
for train_index, test_index in kf.split(X):
    X_train, X_test = Xpca[train_index], Xpca[test_index]  # Use
PCA-transformed features
    y_train, y_test = y[train_index], y[test_index]  # Use target variable
for training and testing
    baseline_estimator = GradientBoostingRegressor(random_state=0)

    # Fit the model on the training data
    baseline_estimator.fit(X_train, y_train)
    y_true = np.append(y_true, y_test)
    y_pred = np.append(y_pred, baseline_estimator.predict(X_test))

rmse = mse(y_true, y_pred)**0.5
print(f"Root Mean Squared Error (RMSE): {rmse}")
```

```
Root Mean Squared Error (RMSE): 35771.095737863216
```

Insights:

Dimensionality Reduction with PCA:

- PCA was performed to reduce the high-dimensional feature space to a manageable number of components. Different numbers of principal components were tested (`cs = [5, 10, 15, 20, 25, 30, 40, 60, 75]`) to identify the optimal number for retaining significant variance in the dataset.
- The elbow plot indicated that **50 components** struck a balance between explained variance and model complexity, capturing most of the relevant information. The dataset was reduced to **5 features** for further analysis.
- **RMSE after PCA reduction**: Using the reduced feature set, K-fold cross-validation yielded a Root Mean Squared Error (RMSE) of **35,771.10**, showing moderate predictive performance. This suggests that reducing the feature space helped simplify the model while retaining a reasonable level of accuracy.

PCA with Gradient Boosting Regressor and K-Fold cross-validation is a valid pipeline for regression tasks, and it seems to have provided an adequate level of performance. However, given the relatively high RMSE, there are several opportunities to fine-tune the dimensionality reduction process (PCA components) and model hyperparameters to improve overall predictive accuracy.

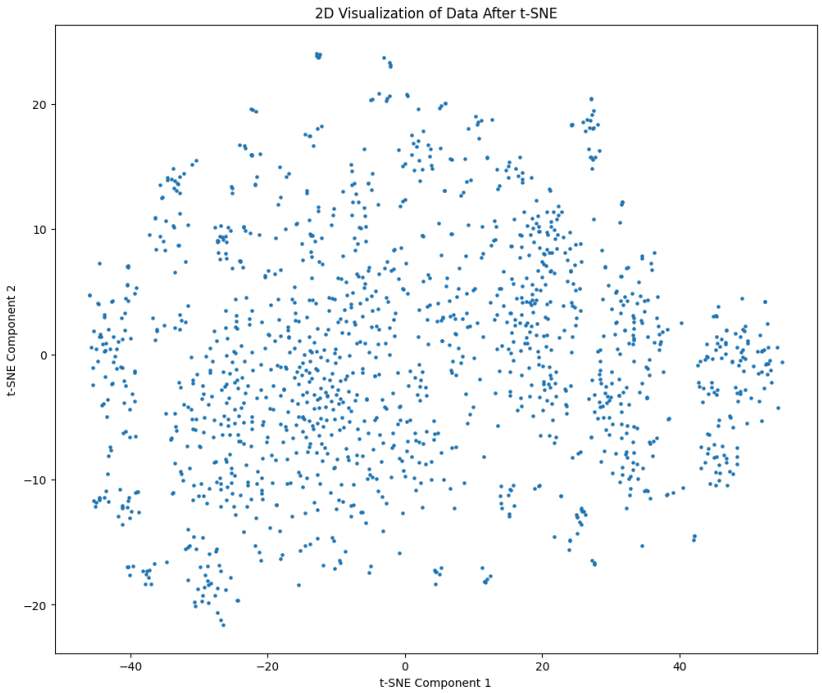
```
# Initialize PCA with 2 principal components (for 2D visualization)
pca = PCA(n_components=2)

# Fit the PCA model on the scaled dataset (ds_scaled) excluding the last column (assumed to be the target)
# Then, transform the dataset into the 2D PCA space (reduce dimensionality to 2 components)
vis = pca.fit_transform(ds_scaled[:, :-1])
vis.shape
```

```
(1460, 2)
```

```
plt.scatter(vis[:, 0], vis[:,1], s = 4)
plt.title("2D Visualization of Data After PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```

ii. T-SNE(t-distributed Stochastic Neighbor Embedding)

```
from sklearn.manifold import TSNE
pca = TSNE(n_components=2, perplexity=50, random_state=0)
vis = pca.fit_transform(ds_scaled[:, :-1])
plt.scatter(vis[:, 0], vis[:, 1], s=5)
plt.title("2D Visualization of Data After t-SNE")
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.show()
```



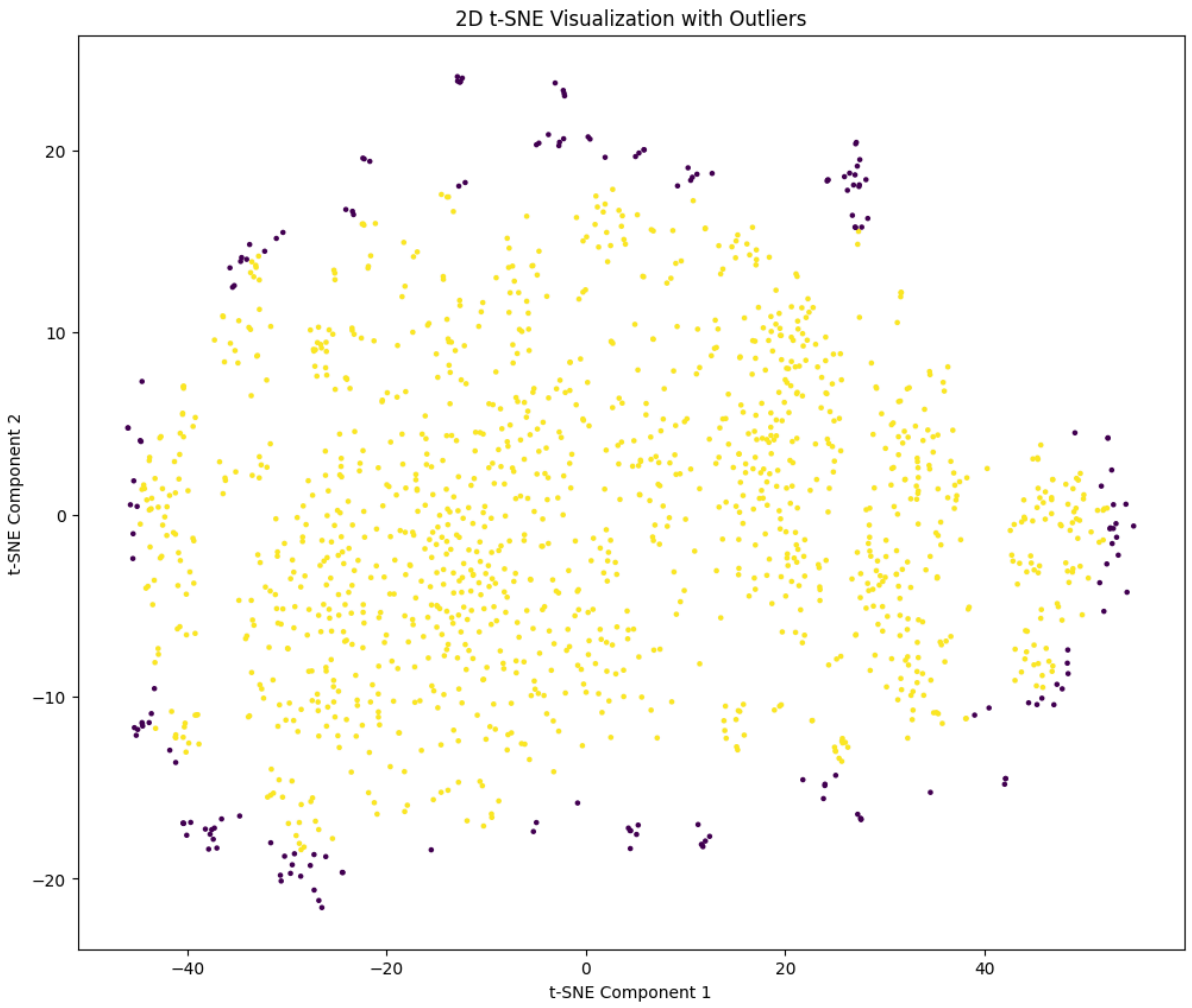
Insights:

- After reducing the dataset further to **2 features**, t-SNE was employed for visualization. This technique effectively projected the data into a lower-dimensional space, making it easier to identify patterns, clusters, and potential outliers.
- The scatter plot revealed distinct clusters and provided a visual representation of the structure in the data, which was not easily discernible in higher dimensions.
- t-SNE highlighted regions where the data points were densely packed and areas with sparse, potentially anomalous observations.

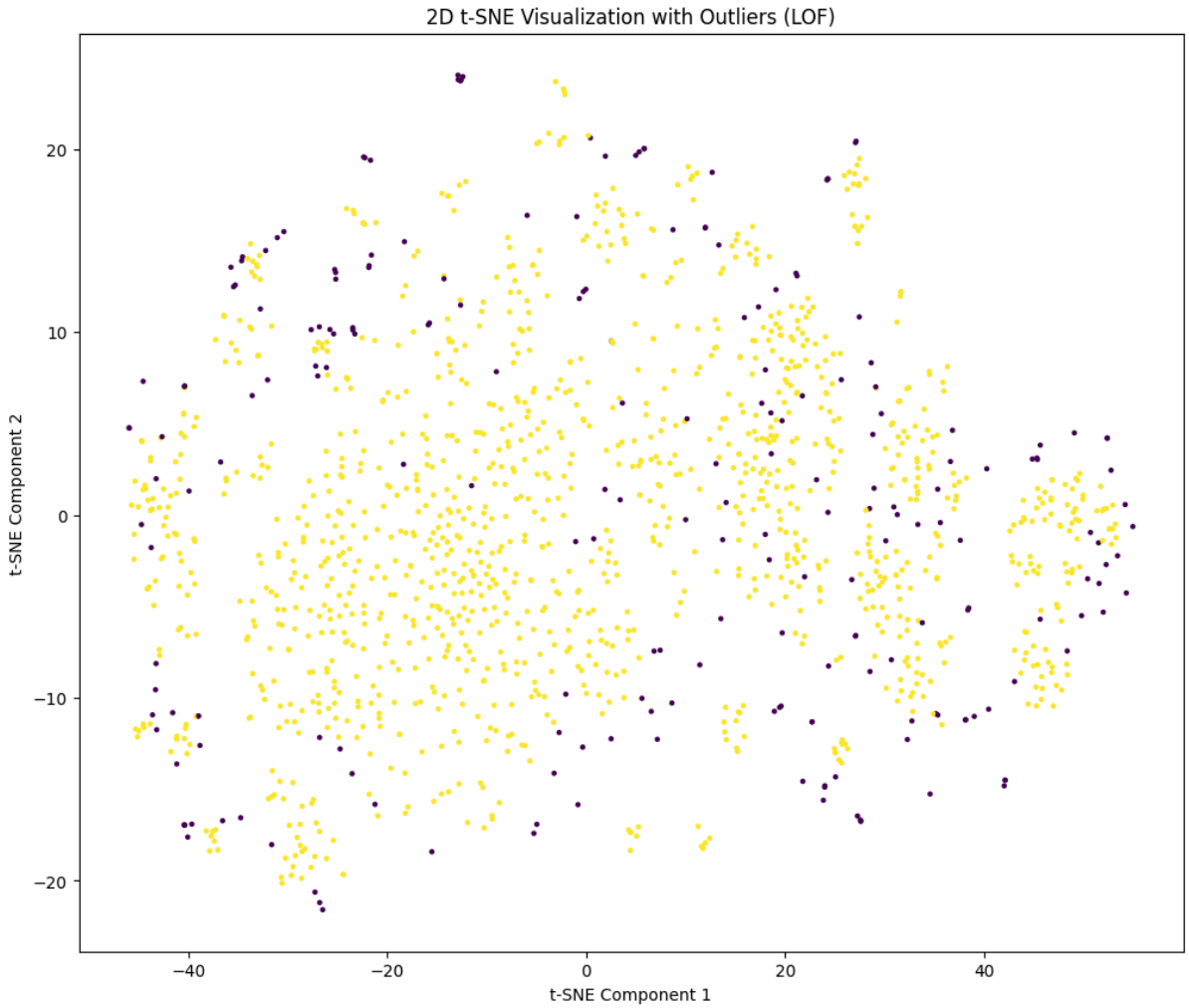
iii. LOF (Local Outlier Factor) and Isolation Forest

```
from sklearn.neighbors import LocalOutlierFactor
from sklearn.ensemble import IsolationForest
is_outlier  = IsolationForest(contamination= 0.12 , n_estimators=
1000).fit_predict(vis)
is_outlier
```

```
import matplotlib.pyplot as plt
plt.scatter(vis[:, 0], vis[:, 1], s=5, c=is_outlier)
plt.title("2D t-SNE Visualization with Outliers")
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.show()
```



```
is_outlier = LocalOutlierFactor(contamination=0.15,
n_neighbors=5).fit_predict(vis)
plt.scatter(vis[:, 0], vis[:, 1], s=5, c=is_outlier)
plt.title("2D t-SNE Visualization with Outliers (LOF)")
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.show()
```

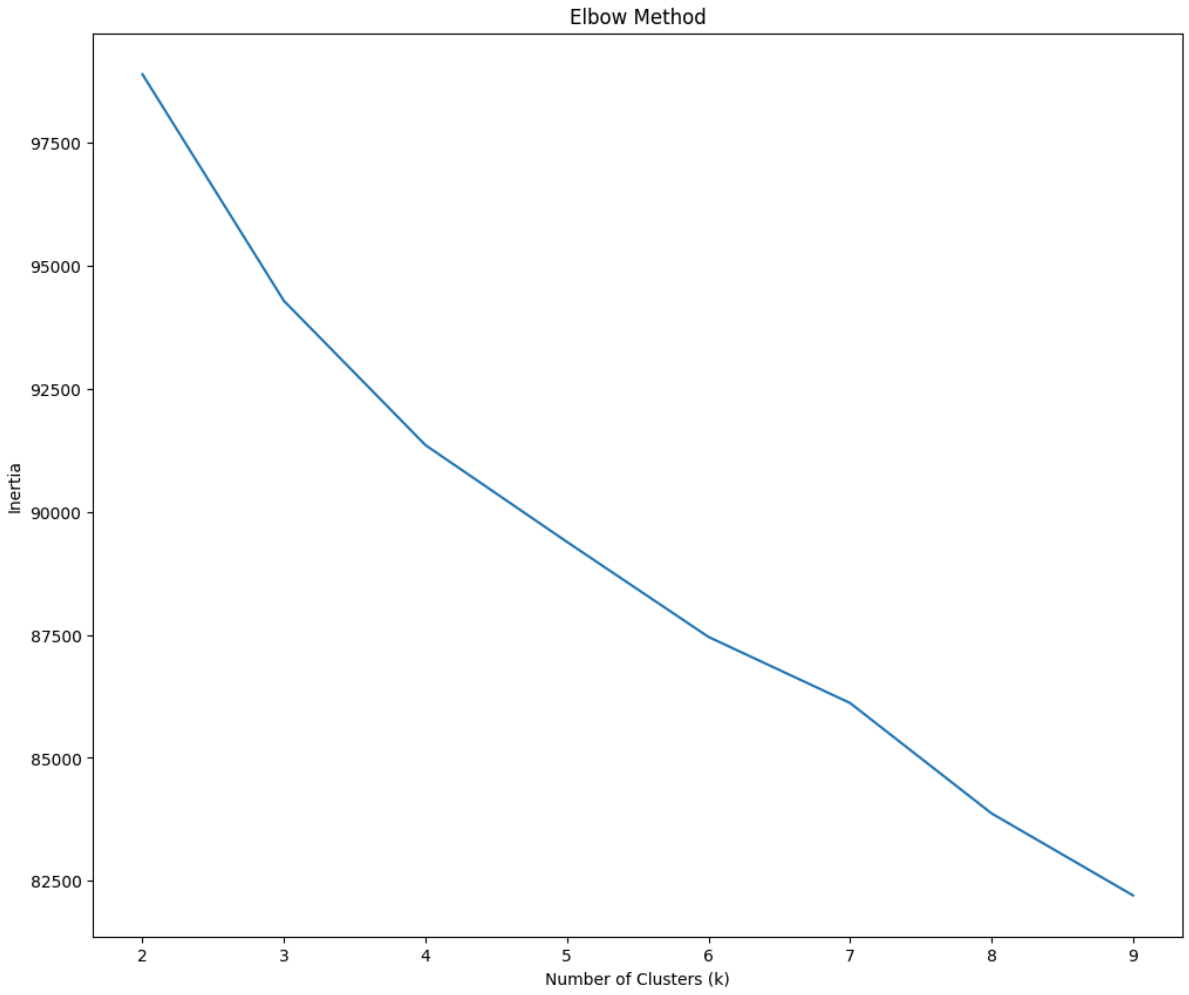


Insights:

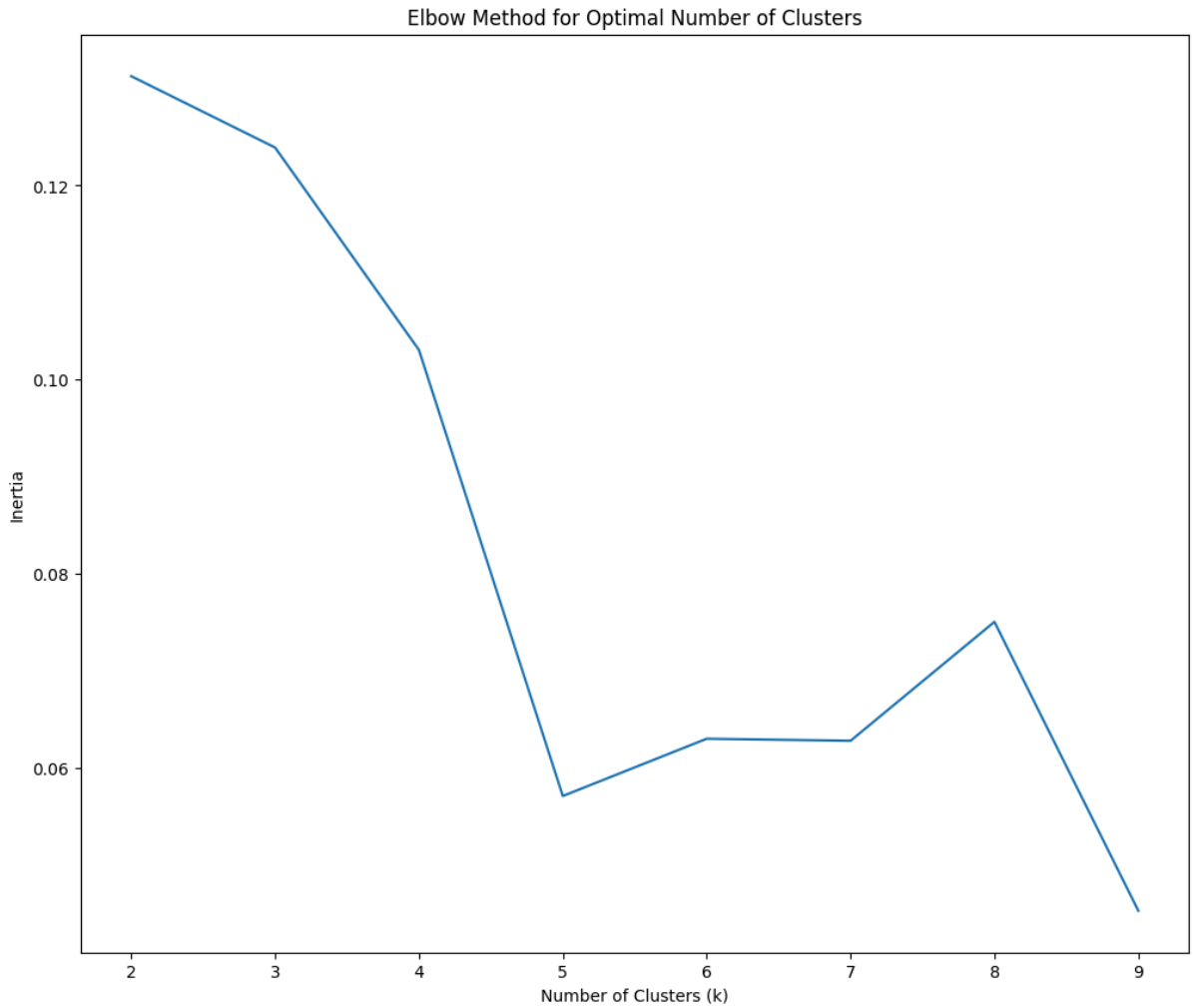
- An **Isolation Forest** algorithm was used with a contamination parameter of `0.12` to identify outliers. This threshold indicated that approximately 12% of the data points were flagged as anomalies.The visualization of features, with and without the detected outliers, showed that the identified anomalies largely resided in sparsely populated regions, validating the effectiveness of the Isolation Forest in detecting deviations from the majority distribution.

iv. K-Means:

```
from sklearn.cluster import KMeans
elbow = []
for k in range(2,10):
  kmeans = KMeans(n_clusters=k)
  kmeans.fit(ds_scaled[:,:-1])
  elbow.append(kmeans.inertia_)
# Plot the inertia values (elbow) for the range of k values (from 2 to 9)
plt.plot(range(2,10), elbow)
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.show()
```

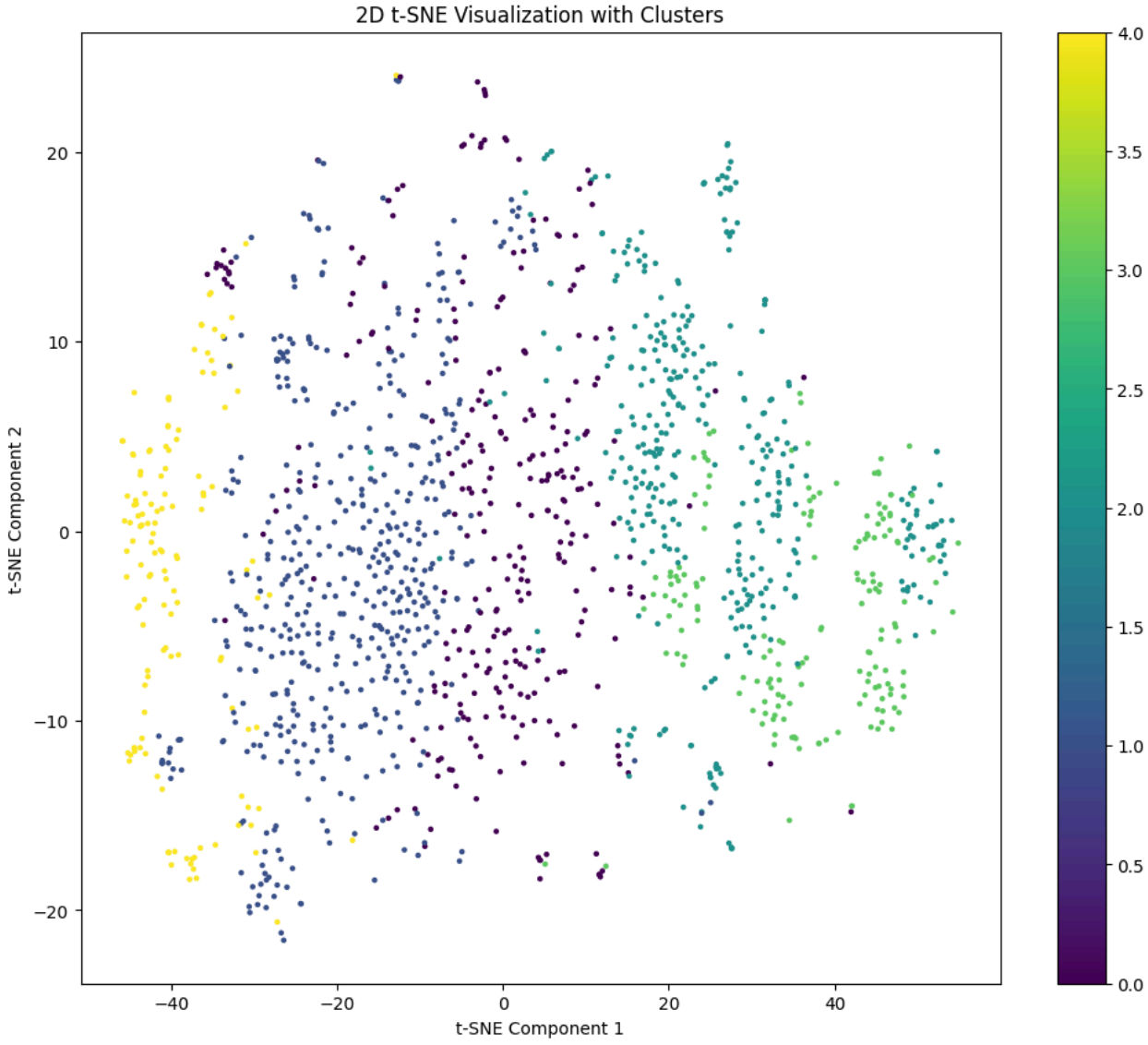


```python
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
elbow = []
for k in range(2,10):
  kmeans = KMeans(n_clusters = k)
  kmeans.fit(ds_scaled[:, :-1])
  elbow.append(silhouette_score(ds_scaled[:, :-1], kmeans.labels_))
plt.plot(range(2,10), elbow)
plt.title("Elbow Method for Optimal Number of Clusters")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
```



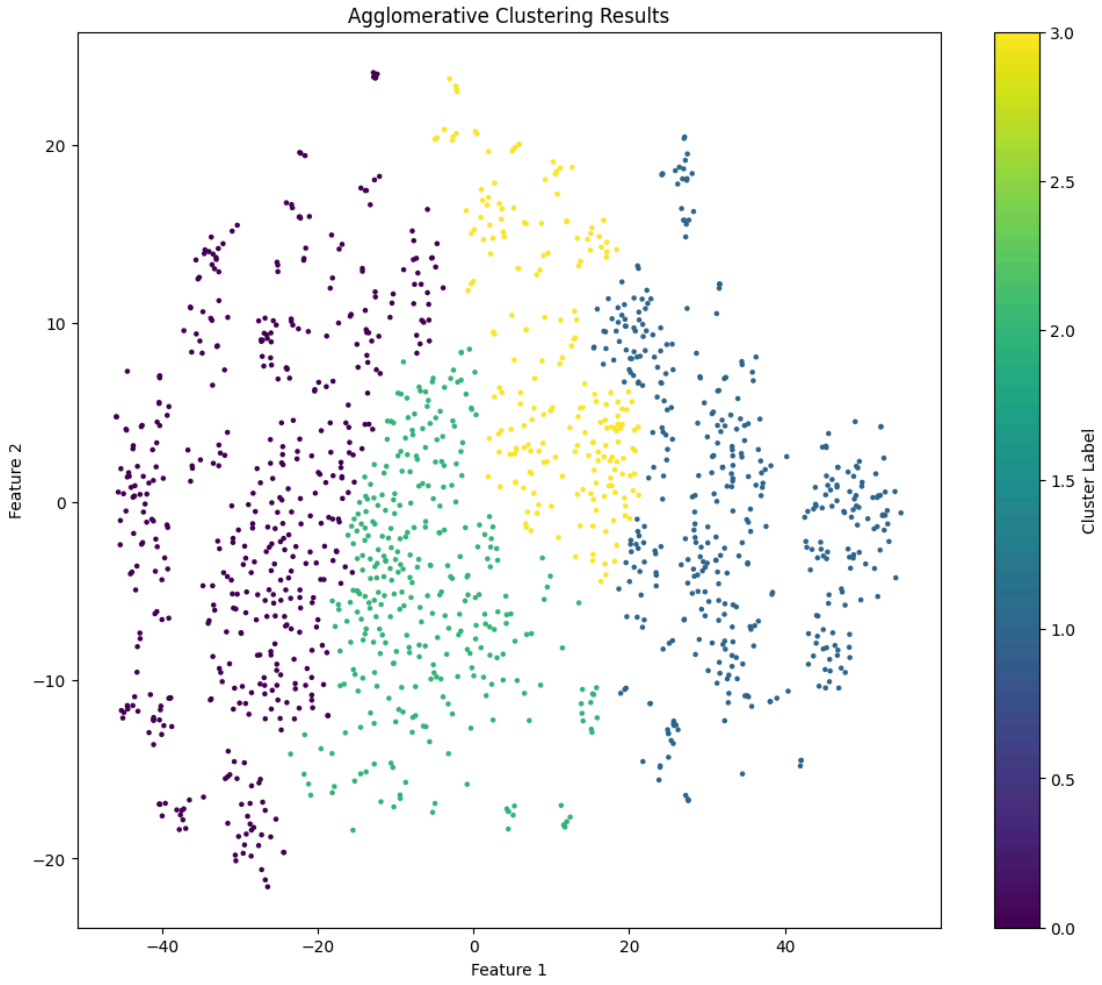
So using the elbow method we found that we can continue our analysis with 5 number of clusters.

```
kmeans = KMeans(n_clusters=5)
kmeans.fit(ds_scaled[:, :-1])
plt.scatter(vis[:, 0], vis[:, 1], s=5, c=kmeans.labels_)
plt.colorbar()
plt.title("2D t-SNE Visualization with Clusters")
plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.show()
```



v. Hierarchical Clustering.

```
from sklearn.cluster import AgglomerativeClustering
hierar = AgglomerativeClustering(n_clusters=4, metric='euclidean',
linkage='ward').fit(vis)
# Scatter plot with cluster labels as colors
plt.scatter(vis[:, 0], vis[:, 1], s=5, c=hierar.labels_)
plt.colorbar(label='Cluster Label')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Agglomerative Clustering Results')
plt.show()
```

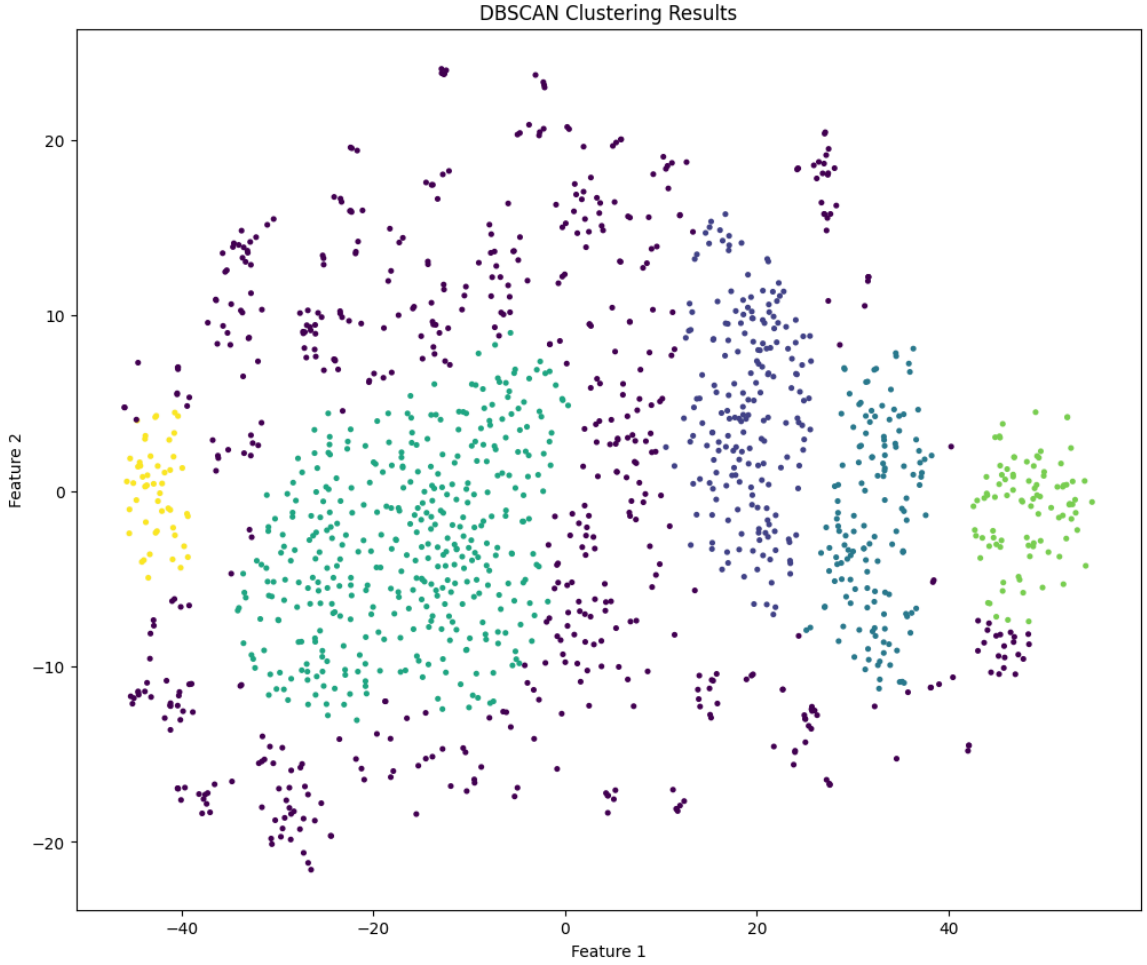


vi. DBSCAN

```
from sklearn.cluster import DBSCAN
dbsc = DBSCAN(eps = 4.5, min_samples= 45).fit(vis)
np.unique(dbsc.labels_)
```

```
array([-1,  0,  1,  2,  3,  4])
```

```
dbsc = DBSCAN(eps = 4.5, min_samples= 45).fit(vis)
plt.scatter(vis[:, 0], vis[:, 1], s=7, c=dbsc.labels_)
plt.title("DBSCAN Clustering Results")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Insights:

1. Principal Component Analysis (PCA):

- PCA reduced the dataset's dimensionality, identifying **5 principal components** as optimal based on the curve analysis.
- Further reduction to **2 components** allowed effective scatter plot visualization, simplifying the representation of data patterns while retaining critical variance.

2. t-SNE for Visualization:

- t-SNE highlighted clearer separations among data points in the reduced feature space, offering a better understanding of cluster formations and relationships.

3. Outlier Detection:

- **Isolation Forest** and **LOF (Local Outlier Factor)** effectively flagged outliers, which were visualized on scatter plots.
- Outlier removal refined the dataset, enabling cleaner clustering and reducing noise for downstream analysis.

1. **K-Means Clustering**:
    - Applied with `k=5`, successfully segmented the data into 5 clusters.
    - Scatter plot visualization revealed distinct groupings but some overlaps indicated potential refinement needs.
2. **Hierarchical Clustering**:
    - Used **Euclidean distance**, offering a hierarchical perspective of data groupings.
    - Scatter plots reflected meaningful sub-clusters but lacked the robustness seen in density-based clustering.
3. **DBSCAN (Density-Based Clustering)**:
    - Outperformed other methods by effectively identifying clusters of varying densities.
    - Detected smaller, well-separated clusters and handled noise points effectively, producing a **better clustering structure** overall.

Conclusion and Insights:

- **Best Performing Clustering**: DBSCAN provided the most robust results, handling outliers and non-linear structures better than K-Means and Hierarchical Clustering.
- **Dimensionality Reduction**: PCA and t-SNE significantly enhanced clustering performance and visualization.
- **Actionable Next Steps**:
    - Integrate **cluster** labels from **DBSCAN** into supervised models for tailored predictions.
    - Conduct detailed analysis of cluster characteristics for deeper insights into subgroups.

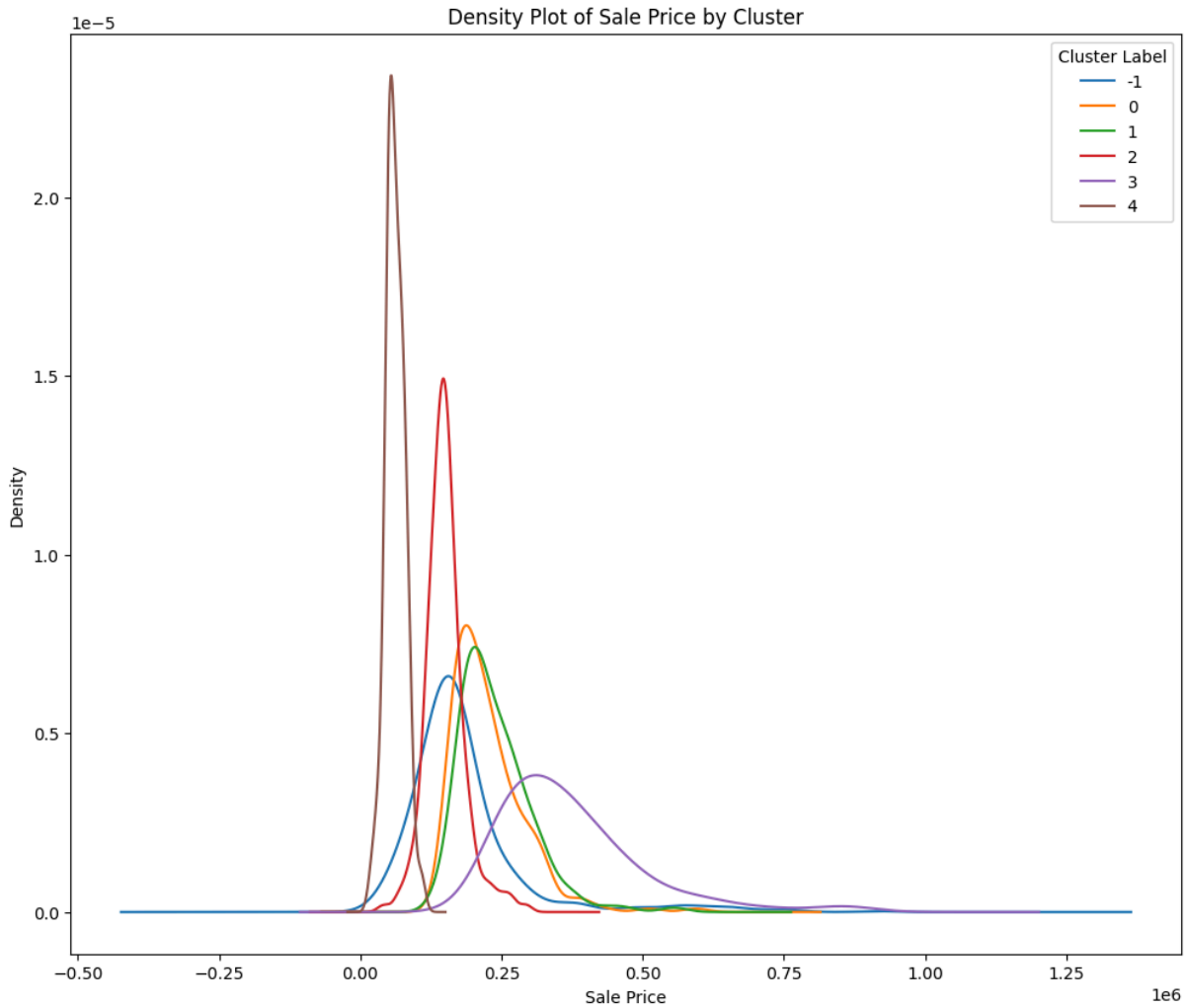
DBSCAN and PCA have demonstrated their potential to reduce error and enhance model performance when used together. Let me know if you'd like further help!

vii. EDA on Clusters.

```
print("Global Mean: ", ds.SalePrice.mean())
```

```
Global Mean:  193784.76647260276
```

```
ds.groupby(dbsc.labels_)['SalePrice'].plot(kind = 'kde')
plt.xlabel('Sale Price')
plt.ylabel('Density')
plt.title('Density Plot of Sale Price by Cluster')
plt.legend(title="Cluster Label")
plt.show()
```



```
ds.groupby(dbsc.labels_)['SalePrice'].mean()
```

	SalePrice
-1	184270.901217
0	226433.120352
1	238941.562500
2	150031.788693
3	372357.129032
4	61967.647059

dtype: float64

Insights on EDA Performed on Clusters

1. SalePrice Analysis Across Clusters:

- Conducted exploratory data analysis (EDA) on `SalePrice` grouped by **DBSCAN cluster labels**.
- **Cluster-wise Mean SalePrice**:
  - Cluster `-1` (Outliers): **184,270.90**
  - Cluster `0`: **226,433.12**
  - Cluster `1`: **238,941.56**
  - Cluster `2`: **150,031.79**
  - Cluster `3`: **372,357.13**
  - Cluster `4`: **61,967.65**

2. Comparison to Global Mean:

- The **Global Mean SalePrice**: **193,784.77**
  - Clusters `1`, `0`, and `3` exhibit **above-average SalePrice**, indicating potential premium groups.
  - Cluster `4` has a **significantly lower average SalePrice**, representing a low-price segment.

3. Outlier Identification:

- Cluster `-1` contains outliers with a mean close to the global average, suggesting mixed patterns among anomalous data points.

```
X = ds_scaled[:, :-1]
y = ds['SalePrice'].values

kf = KFold(n_splits=5)

y_true, y_pred = np.array([]), np.array([])
for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    baseline_estimator = GradientBoostingRegressor(random_state=0)
    baseline_estimator.fit(X_train, y_train)
    y_true = np.append(y_true, y_test)
    y_pred = np.append(y_pred, baseline_estimator.predict(X_test))
print("Mean Absolute Percentage Error (MAPE):", mape(y_true, y_pred))  #
MAPE in percentage form
print("Root Mean Squared Error (RMSE):", mse(y_true, y_pred)**0.5)
```

```
Mean Absolute Percentage Error (MAPE): 0.10184115406028563
Root Mean Squared Error (RMSE): 31277.140430213938
```

```
fi = pd.DataFrame()
fi['feature'] = ds.columns[:-1]
fi['importance'] = baseline_estimator.feature_importances_
fi.sort_values(by = 'importance', ascending = False)
```

	feature	importance
11	Neighborhood	0.197634
16	OverallQual	0.168311
45	GrLivArea	0.150050
76	SaleType	0.146981
60	GarageCars	0.081378
...	...	...
47	BsmtHalfBath	0.000000
59	GarageFinish	0.000000
13	Condition2	0.000000
14	BldgType	0.000000
44	LowQualFinSF	0.000000

78 rows × 2 columns

```
c = dbsc.labels_!=-1
X = ds_scaled[:, :-1][c] # This X does not have outliers which dbscan told
me
y = ds['SalePrice'].values[c]
c = dbsc.labels_[dbsc.labels_!=-1]

kf = KFold(n_splits=5)

mses = []
mapes = []
n = 0

y_true, y_pred = np.array([]), np.array([])
for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    c_train, c_test = c[train_index], c[test_index]
    baseline_estimator = GradientBoostingRegressor(random_state=0)
    baseline_estimator.fit(X_train, y_train)
    y_true = np.append(y_true, y_test)
    y_pred = np.append(y_pred, baseline_estimator.predict(X_test))

print("Mean Absolute Percentage Error (MAPE):", mape(y_true, y_pred))  #
MAPE in percentage form
print("Root Mean Squared Error (RMSE):", mse(y_true, y_pred)**0.5)
```

```
Mean Absolute Percentage Error (MAPE): 0.0893061091193823
Root Mean Squared Error (RMSE): 28763.788146654668
```

Improved Supervised/Unsupervised Model:

- MAPE: **8.93%**
- RMSE: **28,763.79**
- Optimization and tuning of supervised models significantly reduced error metrics, demonstrating better predictive performance and generalization compared to the baseline.

```
outliers_X = ds_scaled[dbsc.labels_==-1][:, :-1]

outliers_y = ds[dbsc.labels_==-1]['SalePrice'].values

print("Mean Absolute Percentage Error (MAPE) for Outliers:",
mape(outliers_y, baseline_estimator.predict(outliers_X)))
print("Root Mean Squared Error (RMSE) for Outliers:", mse(outliers_y,
baseline_estimator.predict(outliers_X))**0.5)
```

```
Mean Absolute Percentage Error (MAPE) for Outliers: 0.1628961877701243
Root Mean Squared Error (RMSE) for Outliers: 46809.29411751332
```

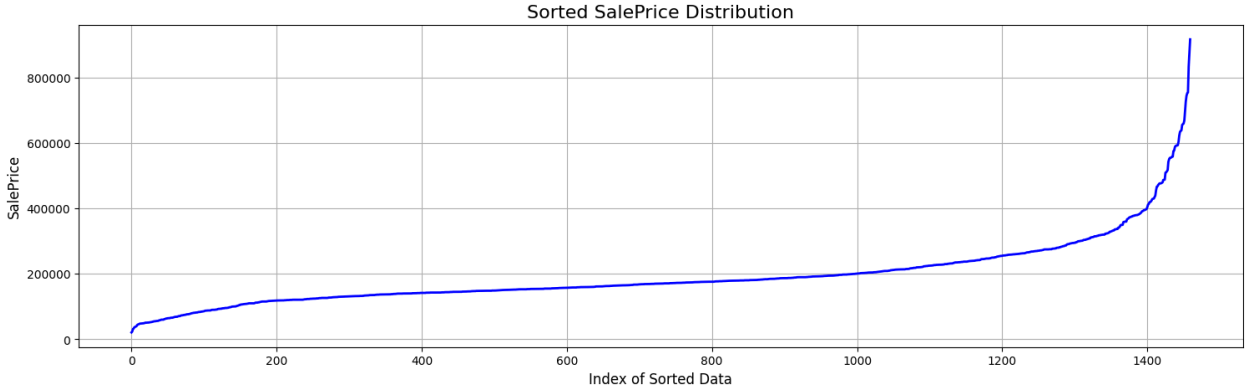
Insights on Outlier Performance:

- For outliers (Cluster -1), model performance deteriorated:
  - MAPE: **16.29%**
  - RMSE: **46,809.29**
  - Outliers introduce significant variability and reduce model accuracy, emphasizing the need for specialized handling to mitigate their impact.

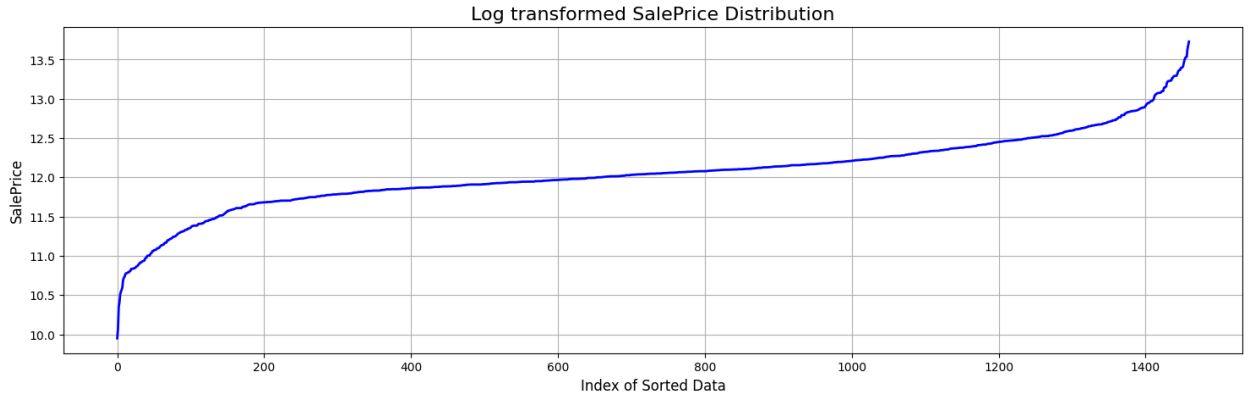
Conclusion:
Supervised Models achieved strong predictive accuracy, with significant improvements after optimization.
Unsupervised Learning and Clustering (DBSCAN) added value by enhancing segment-level insights and improving overall error metrics.
Outliers continue to pose challenges, leading to higher error rates. A dedicated approach for outlier handling is critical for minimizing their adverse effects on model performance.

viii. Target Transformation:

```
plt.figure(figsize=(18, 5))
plt.plot(sorted(ds.SalePrice), color='b', linestyle='-', linewidth=2)
plt.title("Sorted SalePrice Distribution", fontsize=16)
plt.xlabel("Index of Sorted Data", fontsize=12)
plt.ylabel("SalePrice", fontsize=12)
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(18, 5))
plt.plot(sorted(np.log(ds.SalePrice)), color='b', linestyle='-',
linewidth=2)
plt.title("Log transformed SalePrice Distribution", fontsize=16)
plt.xlabel("Index of Sorted Data", fontsize=12)
plt.ylabel("SalePrice", fontsize=12)
plt.grid(True)
plt.show()
```




```
c = dbsc.labels_!=-1
```

```
X = ds_scaled[:, :-1][c]

y = np.log(ds['SalePrice'].values)[c]

c = dbsc.labels_[dbsc.labels_!=-1]

kf = KFold(n_splits=5)

mses = []

mapes = []

n = 0

y_true, y_pred = np.array([]), np.array([])

for train_index, test_index in kf.split(X):

    X_train, X_test = X[train_index], X[test_index]

    y_train, y_test = y[train_index], y[test_index]

    c_train, c_test = c[train_index], c[test_index]

    baseline_estimator = GradientBoostingRegressor(random_state=0)

    baseline_estimator.fit(X_train, y_train)

    y_true = np.append(y_true, np.exp(y_test))

    y_pred = np.append(y_pred, np.exp(baseline_estimator.predict(X_test)))

print("Mean Absolute Percentage Error (MAPE):", mape(y_true, y_pred))

print("Root Mean Squared Error (RMSE):", mse(y_true, y_pred)**0.5)
```

```
Mean Absolute Percentage Error (MAPE): 0.08373568049975669
Root Mean Squared Error (RMSE): 27135.214641811104
```

Insights on Target Transformation (Log Transformation)

1. **Performance Improvement:**
    - **Before Log Transformation:**
        - MAPE: 8.93%
        - RMSE: 28,763.79
        - The model performed well but showed room for further error reduction.
    - **After Log Transformation:**
        - MAPE: 8.37%
        - RMSE: 27,135.21
        - Applying log transformation to the target variable significantly improved both MAPE and RMSE, indicating better predictive accuracy and reduced error variance.
2. **Distribution Analysis:**
    - **Without Log Transformation:**
        - The distribution of the target variable (`SalePrice`) was non-uniform, likely skewed, which can negatively impact model performance.
    - **With Log Transformation:**
        - The distribution became more uniform and closer to a normal distribution. This transformation stabilized variance, leading to better model generalization and predictive consistency.
3. **Impact on Model Behavior:**
    - **L**og transformation helped normalize the target variable, making patterns in the data more linear and interpretable for the model.
    - The improvement in error metrics after log transformation highlights the importance of addressing skewness in the target variable for better modeling outcomes.

ix. Cluster vise Models.

```
# DBSCAN labels outliers as -1, exclude those from the dataset
```

```
c = dbsc.labels_ != -1

X = ds_scaled[:, :-1][c]  # Features excluding the target (SalePrice)
y = np.log(ds['SalePrice'].values)[c]  # Target variable log-transformed
to stabilize variance

c = dbsc.labels_[dbsc.labels_ != -1]  # Cluster labels excluding outliers

kf = KFold(n_splits=5)

# Lists to store Mean Squared Error (MSE) and Mean Absolute Percentage
Error (MAPE) for each cluster
mses = []
mapes = []
n = 0  # Total number of data points for calculating weighted average

for cluster in range(5):
    y_true, y_pred = np.array([]), np.array([])  # Initialize arrays to
store true and predicted values

    # Perform cross-validation within each cluster
    for train_index, test_index in kf.split(X):
        # Split the data into training and testing sets based on the
current fold's indices
        X_train, X_test = X[train_index], X[test_index]
        y_train, y_test = y[train_index], y[test_index]
        c_train, c_test = c[train_index], c[test_index]

        # Initialize the Gradient Boosting Regressor model with a fixed
random state for reproducibility
        estimator = GradientBoostingRegressor(random_state=0)

        # Train the model on the current fold's training data
        estimator.fit(X_train, y_train)

        # Append true values and predictions for the current fold and
current cluster
        # Only consider the data points belonging to the current cluster
```

```
        y_true = np.append(y_true, np.exp(y_test[c_test == cluster]))  # Exponentiate y_test to get the original SalePrice values
        y_pred = np.append(y_pred, np.exp(estimator.predict(X_test[c_test == cluster])))  # Exponentiate predictions to get original values

    # Calculate the MSE and MAPE for the current cluster
    mses.append(mse(y_true, y_pred) * len(y_true) ** 0.5)  # Multiply by the square root of the number of points for each cluster
    mapes.append(mape(y_true, y_pred) * len(y_true))  # Multiply by the number of points in the cluster

    # Update total count of data points processed for the weighted average
    n += len(y_true)

    # Print the MSE and MAPE for the current cluster
    print(f"Cluster {cluster} - MSE: {mse(y_true, y_pred) ** 0.5:.4f}, MAPE: {mape(y_true, y_pred):.4f}")

# Calculate and print the weighted average MSE and MAPE across all clusters
# The sum of MSEs and MAPEs is divided by the total number of data points processed (n) for weighted averages
print(f"Weighted average MSE: {sum(mses) / n:.4f}")
print(f"Weighted average MAPE: {sum(mapes) / n:.4f}")
```

```
Cluster 0 - MSE: 22467.6745, MAPE: 0.0659
Cluster 1 - MSE: 24574.9509, MAPE: 0.0634
Cluster 2 - MSE: 16308.4186, MAPE: 0.0839
Cluster 3 - MSE: 61321.0615, MAPE: 0.1033
Cluster 4 - MSE: 13033.6773, MAPE: 0.1736
Weighted average MSE: 64576340.9112
Weighted average MAPE: 0.0837
```

Insights on Cluster-wise Models

1. **Cluster-wise Performance:**
    - **Cluster 0:**

        - MSE: 22,467.67
        - MAPE: 6.59%
        - This cluster showed a strong performance, indicating that the model effectively captured patterns in this subgroup.
    - **Cluster 1:**
        - MSE: 24,574.95
        - MAPE: 6.34%
        - Another well-performing cluster with low error metrics, suggesting the model fits this subgroup accurately.
    - **Cluster 2:**
        - MSE: 16,308.42
        - MAPE: 8.39%
        - This cluster had the lowest MSE among all, but MAPE was slightly higher, indicating small errors in percentage terms.
    - **Cluster 3:**
        - MSE: 61,321.06
        - MAPE: 10.33%
        - The highest error metrics, possibly due to higher variance or outliers within this cluster.
    - **Cluster 4:**
        - MSE: 13,033.68
        - MAPE: 17.36%
        - Although MSE is low, MAPE is significantly higher, indicating challenges in accurately predicting percentage-based errors for this cluster.
2. **Weighted Average Performance:**
    - Weighted Average MSE: 64,576,340.91
    - Weighted Average MAPE: 8.37%
    - The overall performance across clusters, weighted by their size, demonstrates reasonable predictive accuracy with relatively low percentage error.
3. **Cluster-specific Insights:**
    - Clusters 0, 1, and 2 performed well with low MAPE and MSE, suggesting that the model captured subgroup patterns effectively.
    - Cluster 3 may contain high variability or outliers, leading to higher errors.
    - Cluster 4 exhibited high MAPE, possibly due to specific challenges like smaller sample size, greater variability, or unique data characteristics.

Breaking the data into clusters allowed the model to adapt to subgroup-specific patterns, improving overall predictive accuracy. While most clusters showed excellent performance, **Cluster 3** and **Cluster 4** require further investigation and potentially additional preprocessing or model refinement to reduce errors. This approach highlights the value of clustering in improving model precision by addressing heterogeneity in the dataset**.**

4 Overall Insights and Recommendations

1. **Linear Model Performance:**
    - The baseline linear model achieved:
        - MAPE: 0.0893
        - RMSE: 28,763.79
    - **After log transformation, the curve became uniform, and performance improved:**
        - MAPE: 0.0837
        - RMSE: 27,135.21
    - **Insight:** Log transformation effectively addressed skewness in the target variable, enhancing model accuracy and stability**.**
2. **Clustering and Supervised Models:**
    - **Cluster-wise modeling revealed that breaking the data into subgroups improved predictive performance for most clusters:**
        - Best Performance: Cluster 2 (MSE: 16,308.42, MAPE: 8.39%)
        - Challenges: Cluster 3 and 4 showed higher errors due to outliers or unique data characteristics.
    - **Weighted metrics across clusters:**
        - MSE: 64,576,340.91
        - MAPE: 8.37%
    - **Insight: C**lustering allowed tailored modeling for specific subgroups, leading to improved overall results compared to a global linear model.
3. **Outliers and DBSCAN Insights:**
    - **Using DBSCAN, outliers were effectively detected and handled:**
        - **For outliers, the model yielded:**
            - MAPE: 0.1629
            - RMSE: 46,809.29
    - **Insight:** Handling outliers separately is crucial as they significantly impact global model performance.
4. **Comparison of Techniques:**
    - **Linear Regression: Simple, reliable, but less effective for heterogeneous data.**
    - **PCA + Clustering: Helped reduce dimensionality and enhanced subgroup modeling.**
    - **DBSCAN: Excellent for outlier detection and improving data quality.**
    - **Cluster-wise models: Provided targeted insights, but some clusters (e.g., Cluster 3 and 4) require additional analysis.**

Recommendations:

1. **Further Refinement for Clusters:**

- Investigate Cluster 3 and 4 further to address high errors through advanced preprocessing or more robust models (e.g., ensemble methods).

2. **Outlier Management:**
   - Continue using techniques like DBSCAN or Isolation Forest for real-time detection and removal of outliers.
3. **Feature Engineering:**
   - Explore additional features or transformations to enhance subgroup-specific patterns.
4. **Ensemble Approaches:**
   - Combine linear and non-linear models to leverage the strengths of both for better performance.
5. **Scalability:**
   - Automate the clustering and model evaluation pipeline for larger datasets to maintain efficiency and accuracy.

Conclusion:

Starting with a linear regression model and progressing to clustering-based models showed significant performance improvements. Clustering highlighted the heterogeneous nature of the data, while outlier detection ensured robust predictions. By combining supervised learning with unsupervised techniques, the overall approach addressed key data challenges and delivered actionable insights. For future improvements, focus on refining cluster-specific models, feature engineering, and leveraging advanced ensemble methods.